

BlumGoldwasser

aysengurdal

January 2, 2025

Contents

theory *BG-aux*
 imports *Sigma-Commit-Crypto.Number-Theory-Aux*
begin

1 Auxillary Lemmas

Here, we have all the auxillary definitions used in the formalization of Blum-Goldwasser cryptosystem

2 Auxillary Lemmas

Here, we have all the auxillary lemmas used to prove the correctness of Blum-Goldwasser cryptosystem

Our proof of correctness will only consider x -i that are squared in modulo residue of p . We Therefore prove that case only:

lemma *eulers-criterion*:

assumes *prime* ($p :: \text{nat}$) *odd* p *coprime* p x ($\exists y. [y^2 = x] \pmod{p}$)

shows $[x^{(p-1) \text{ div } 2} = 1] \pmod{p}$

proof –

from *assms*(2) **have** *even-p-minus-one*: *even* ($p - 1$) **by** *auto*

then have ($\exists k. p - 1 = 2 * k$) **by** *blast*

then obtain k **where** *obtain-k*: $p - 1 = 2 * k$ **by** *blast*

from *assms*(1) *assms*(3) **have** $\neg p \text{ dvd } x$ **by** *auto*

from *this* *assms*(1) *fermat-theorem*

have $[x^{(p-1)} = 1] \pmod{p}$

by *algebra*

then have $[x^{(p-1) \text{ div } 2} * x^{(p-1) \text{ div } 2} = 1] \pmod{p}$

using *dvd-div-mult-self* *even-p-minus-one* *power2-eq-square* *power-mult*

by (*metis*)

then have *trial*: $[x^{(p-1) \text{ div } 2} = 1] \pmod{p} \vee [x^{(p-1) \text{ div } 2} = -1] \pmod{p}$

```

    using assms(1) cong-int-iff cong-square int-ops(2) mult-0-right nat-code(2)
nat-int
    negative-zle of-nat-mult prime-nat-int-transfer
    by (smt (z3) )

from assms(4) obtain y where obtain-y:  $[y^2 = x] \pmod p$  by blast
from trial obtain-y have options:  $[(y^2) \wedge ((p - 1) \text{ div } 2) = 1] \pmod p \vee$ 
 $[(y^2) \wedge ((p - 1) \text{ div } 2) = -1] \pmod p$ 
using cong-int-iff cong-pow cong-trans
by (meson)
have  $\neg [(y^2) \wedge ((p - 1) \text{ div } 2) = -1] \pmod p$ 
proof (rule ccontr)
  assume  $\neg \neg [(y^2) \wedge ((p - 1) \text{ div } 2) = -1] \pmod p$ 
  then have  $[(y^2) \wedge ((p - 1) \text{ div } 2) = -1] \pmod p$  by auto
  then have  $[(y) \wedge ((p - 1) \text{ div } 2) * 2 = -1] \pmod p$  using power-mult
  by (metis mult.commute)
  then have y-minus-one:  $[(y) \wedge (p - 1) = -1] \pmod p$ 
  using even-p-minus-one by auto

have  $\neg p \text{ dvd } y$ 
using  $\langle \neg p \text{ dvd } x \rangle$  assms(1) cong-dvd-iff obtain-y pos2 prime-dvd-power-nat-iff
by blast

then have  $[(y) \wedge (p - 1) = 1] \pmod p$ 
using fermat-theorem assms(1) by auto
from this y-minus-one have contradiction:  $[(y) \wedge (p - 1) = -1] \pmod p \wedge$ 
 $[(y) \wedge (p - 1) = 1] \pmod p$  by auto

then have  $[-1 = (y) \wedge (p - 1)] \pmod p \wedge [(y) \wedge (p - 1) = 1] \pmod p$ 
using cong-sym-eq by auto
then have  $[-1 = (y) \wedge (p - 1)] \pmod p \wedge [\text{int } (y) \wedge (p - 1) = \text{int } 1] \pmod p$ 
using cong-int-iff int-ops(2) by force
then have  $[-1 = 1] \pmod p$  using cong-trans by auto
then have  $[1 = -1] \pmod p$  using cong-sym by auto
then have  $p \text{ dvd } (1 - (-1))$  using cong-iff-dvd-diff
by fast
then have  $p \text{ dvd } 2$  by presburger
thus False
using assms(2) assms(1) primes-dvd-imp-eq two-is-prime-nat by blast
qed

from this options have  $[(y^2) \wedge ((p - 1) \text{ div } 2) = 1] \pmod p$  by auto
thus ?thesis using obtain-y cong-pow cong-sym cong-trans
by (meson)
qed

end
theory BG-formalization imports
Main

```

BG-aux
CryptHOL.CryptHOL
Crypto-Standards.Words
begin

3 Introduction

In this theory, our goal is to formalize the Blum-Goldwasser cryptosystem in a deterministic setting. Hence, all the variables that are to be sampled from an oracle are given to the following functions as parameters.

4 Key generation

The key generation process normally picks out two positive integers congruent to 3 mod 4 (where these two integers are the private key) and multiplies the two to create the public key n .

definition *key-gen* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow (\text{nat} \times (\text{nat} \times \text{nat}))$ **where**
key-gen $p\ q = (p * q, (p, q))$

5 Encryption

Before formalizing the encryption process, we define our bitstrings as follows:

type-synonym *bitstring* = *bool list*

This means that we represent bitstrings as a list of boolean values, True for 1 and False for 0. The bitstrings are represented from most significant to least significant in the list (i.e. $44 = 101011 = [\text{True}, \text{False}, \text{True}, \text{False}, \text{True}, \text{True}]$).

We then need a function to split our plaintext message into h -bit blocks. We define the following function for this reason:

fun *split* :: *bitstring* $\Rightarrow$ $\text{nat} \Rightarrow$ *bitstring list* **where**
split [] $n = []$ |
split $m\ 0 = []$ |
split $m\ n = (\text{take } n\ m) \# (\text{split } (\text{drop } n\ m)\ n)$

The function takes the first n elements from the message, puts it to the head of the resulting list, and goes on with splitting the rest.

We define the loop in the encryption as a recursive function. This function takes the nat-to-bit conversion function as a parameter:

fun *enc-loop-func*
 :: $((\text{nat} \Rightarrow \text{bitstring}) \times \text{nat} \times \text{nat} \times \text{bitstring list}) \Rightarrow \text{bitstring list} \Rightarrow \text{nat} \Rightarrow$
 $(\text{bitstring list} \times \text{nat})$

where

```

enc-loop-func (f, n, h, []) c-acc x = (rev c-acc, x) |
enc-loop-func (f, n, h, (m-i # m-rest)) c-acc x =
  (let x-i = x * x mod n in
   let p-i = f (x-i mod 2^h) in
   let c-i = xor-list m-i p-i in
   enc-loop-func (f, n, h, m-rest) (c-i # c-acc) x-i
  )

```

fun *enc-loop-func-between*

```

:: ((nat => bitstring) × nat × nat × bitstring list) ⇒ bitstring list ⇒ nat ⇒
bitstring list

```

where

```

enc-loop-func-between (f, n, h, []) c-acc x = rev c-acc |
enc-loop-func-between (f, n, h, (m-i # m-rest)) c-acc x =
  (let x-i = x * x mod n in
   let p-i = f (x-i mod 2^h) in
   let c-i = xor-list m-i p-i in
   enc-loop-func-between (f, n, h, m-rest) (c-i # c-acc) x-i
  )

```

fun *enc-loop-func-list*

```

:: ((nat => bitstring) × nat × nat × bitstring list) ⇒ nat ⇒ bitstring list

```

where

```

enc-loop-func-list (f, n, h, []) x = [] |
enc-loop-func-list (f, n, h, (m-i # m-rest)) x =
  (let x-i = x * x mod n in
   let p-i = f (x-i mod 2^h) in
   let c-i = xor-list m-i p-i in
   c-i # enc-loop-func-list (f, n, h, m-rest) x-i
  )

```

fun *enc-loop-func-x*

```

:: ((nat => bitstring) × nat × nat × nat) ⇒ nat ⇒ nat

```

where

```

enc-loop-func-x (f, n, h, t) x = x ^ (2 ^ t) mod n

```

A simple function to convert a bit represented by a natural number to a bit represented by a boolean value:

fun *nat-to-bit* :: nat ⇒ bool **where**

```

case-0: nat-to-bit 0 = False |

```

case-1: nat-to-bit (Suc 0) = True |
case-greater: nat-to-bit (Suc (Suc n)) = False

The function gives False for anything other than True.

We finally define a function to convert a natural number to a bitstring, with the help of `nat_to_bits`.

definition *nat-to-bitstring* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *bitstring* **where**
nat-to-bitstring = (λl *n*. *map* (*nat-to-bit*) (*nat-to-bits-len* *n l*))

Abstracting away from accumulators and conversion functions:

fun *enc-loop*
 :: (*nat* $\times$ *nat* $\times$ *bitstring list*) $\Rightarrow$ *nat* $\Rightarrow$ (*bitstring list* $\times$ *nat*)
where
enc-loop (*n*, *h*, *m*) *x*
 = *enc-loop-func* (*nat-to-bitstring* *h*, *n*, *h*, *m*) [] *x*

fun *enc-loop-alt*
 :: (*nat* $\times$ *nat* $\times$ *bitstring list*) $\Rightarrow$ *nat* $\Rightarrow$ (*bitstring list* $\times$ *nat*)
where
enc-loop-alt (*n*, *h*, *m*) *x*
 = (*enc-loop-func-list* (*nat-to-bitstring* *h*, *n*, *h*, *m*) *x*,
enc-loop-func-x (*nat-to-bitstring* *h*, *n*, *h*, *length m*) *x*)

At last, the encryption function:

definition *encrypt* :: *nat* $\Rightarrow$ *bitstring* $\Rightarrow$ *nat* $\Rightarrow$ (*bitstring list* $\times$ *nat*) **where**
encrypt *n m r* = (
 let *h* = *nat* $\lfloor \log 2 (\log 2 n) \rfloor$ in
 let *m-l* = *split m h* in
 let (*x-0* :: *nat*) = (*r* * *r*) mod *n* in
 let (*c*, *x-t*) = *enc-loop* (*n*, *h*, *m-l*) *x-0* in
 let *x-t'* = *x-t* * *x-t* mod *n* in
 (*c*, *x-t'*))

definition *encrypt-alt* :: *nat* $\Rightarrow$ *bitstring* $\Rightarrow$ *nat* $\Rightarrow$ (*bitstring list* $\times$ *nat*) **where**
encrypt-alt *n m r* = (
 let *h* = *nat* $\lfloor \log 2 (\log 2 n) \rfloor$ in
 let *m-l* = *split m h* in
 let (*x-0* :: *nat*) = (*r* * *r*) mod *n* in
 let (*c*, *x-t*) = *enc-loop-alt* (*n*, *h*, *m-l*) *x-0* in
 let *x-t'* = *x-t* * *x-t* mod *n* in
 (*c*, *x-t'*))

An alternate definition of floor log:

fun *floor-nat-log-rec* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat* **where**
floor-nat-log-rec 0 *n acc* = 0 |
floor-nat-log-rec (*Suc* 0) *n acc* = 0 |
floor-nat-log-rec *base* 0 0 = 0 |

```

floor-nat-log-rec base 0 (Suc acc) = acc |
floor-nat-log-rec base (Suc n) acc
  = floor-nat-log-rec base ((Suc n) div base) (Suc acc)

```

definition *floor-nat-log* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat* **where**
floor-nat-log base *n* = *floor-nat-log-rec* base *n* 0

definition *encrypt-discrete-log* :: *nat* $\Rightarrow$ *bitstring* $\Rightarrow$ *nat* $\Rightarrow$ (*bitstring list* $\times$ *nat*)
where

```

encrypt-discrete-log n m r = (
let h = (floor-nat-log 2 (floor-nat-log 2 n)) in
let m-l = split m h in
let (x-0 :: nat) = (r * r) mod n in
let (c, x-t) = enc-loop (n, h, m-l) x-0 in
let x-t' = x-t * x-t mod n in
(c, x-t'))

```

now the decryption...

fun *calculate-exponent* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat* **where**
calculate-exponent t p =
 (let d-p = ((Suc p) div 4) ^ (Suc t) mod (p - 1) in d-p)

fun *merge-list* :: 'a list list $\Rightarrow$ 'a list **where**
merge-list [] = [] |
merge-list (x # xs) = x @ (*merge-list* xs)

definition *decrypt* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ (*bitstring list* $\times$ *nat*) $\Rightarrow$ *bitstring* **where**
decrypt p q tup = (case tup of (c, x) $\Rightarrow$
 (let u-p = x ^ (calculate-exponent (length c) p) mod p in
 let u-q = x ^ (calculate-exponent (length c) q) mod q in
 let ((r-p, r-q), -) = euclid-ext (int p) (int q) in
 let x-0 = nat ((int u-p * r-p * int p + int u-p * r-q * int q) mod (p * q)) in
 let (m, x') = enc-loop (p * q, (nat [log 2 (log 2 (p * q))]) , c) x-0 in
 merge-list m))

definition *decrypt-alt* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ (*bitstring list* $\times$ *nat*) $\Rightarrow$ *bitstring* **where**
decrypt-alt p q tup = (case tup of (c, x) $\Rightarrow$
 (let (u-p::nat) = x ^ (calculate-exponent (length c) p) mod p in
 let (u-q::nat) = x ^ (calculate-exponent (length c) q) mod q in
 let ((r-p, r-q), -) = euclid-ext (int p) (int q) in
 let x-0 = nat ((u-q * r-p * p + u-p * r-q * q) mod int (p * q)) in
 let (m, x') = enc-loop-alt (p * q, (nat [log 2 (log 2 (p * q))]) , c) x-0 in
 merge-list m))

end
theory *BG-correctness* **imports**
Main
BG-formalization
begin

Correctness proof

lemma *l-set-less-h*: $l \in \text{set } (\text{split } m \ h) \implies \text{length } l \leq h$

proof (*induction m h rule: split.induct*)

case (1 n)

then show ?case **by** auto

next

case (2 v va)

then consider (*m-empty*) $m = []$ | (*m-not-empty*) $m \neq []$ **by** auto

then show ?case

proof *cases*

case *m-empty*

then show ?thesis **using** 2 **by** auto

next

case *m-not-empty*

then have $\exists x \ xs. m = x \# xs$ **by** (*auto iff: neq-Nil-conv*)

then show ?thesis **using** 2 **by** (*auto*)

qed

next

case (3 x xs v)

have $\text{split } (x \# xs) \ (\text{Suc } v) = (\text{take } (\text{Suc } v) \ (x \# xs)) \# (\text{split } (\text{drop } (\text{Suc } v) \ (x \# xs)) \ (\text{Suc } v))$

using *split.elims*

by auto

then have $l \in \text{set } ((\text{take } (\text{Suc } v) \ (x \# xs)) \# (\text{split } (\text{drop } (\text{Suc } v) \ (x \# xs)) \ (\text{Suc } v)))$

using 3(2) **by** auto

then have $l = \text{take } (\text{Suc } v) \ (x \# xs) \vee l \in \text{set } (\text{split } (\text{drop } (\text{Suc } v) \ (x \# xs)) \ (\text{Suc } v))$ **by** auto

then consider $l = (\text{take } (\text{Suc } v) \ (x \# xs))$ | $l \in \text{set } (\text{split } (\text{drop } (\text{Suc } v) \ (x \# xs)) \ (\text{Suc } v))$ **by** blast

then show $\text{length } l \leq \text{Suc } v$

proof *cases*

case 1

then show ?thesis **by** auto

next

case 2

then show ?thesis **using** 3(1) **by** auto

qed

qed

corollary *split-l*: $\forall l \in \text{set } (\text{split } m \ h). \text{length } l \leq h$

using *l-set-less-h* **by** auto

lemma *x-exp*:

assumes $p \bmod 4 = 3 \ q \bmod 4 = 3$

prime ($p::\text{nat}$) *prime* q

coprime $x \ (p * q) \ \exists r. [x = r * r] \ (\text{mod } p * q)$

shows $[(x \wedge (2 \wedge (t + 1))) \wedge (((p + 1) \text{ div } 4) \wedge (t + 1)) = x] \pmod{p}$
proof (*induction t*)
case 0
have $(x \wedge (2 \wedge (0 + 1))) \wedge (((p + 1) \text{ div } 4) \wedge (0 + 1))$
 $= (x \wedge (2 \wedge 1)) \wedge (((p + 1) \text{ div } 4) \wedge 1)$ **by** *auto*
then have $(x \wedge (2 \wedge (0 + 1))) \wedge (((p + 1) \text{ div } 4) \wedge (0 + 1))$
 $= (x \wedge 2) \wedge ((p + 1) \text{ div } 4)$ **by** *auto*

then have *x-mod-1*: $(x \wedge (2 \wedge (0 + 1))) \wedge (((p + 1) \text{ div } 4) \wedge (0 + 1))$
 $= x \wedge (2 * ((p + 1) \text{ div } 4))$
using *power-mult[symmetric]* **by** *auto*
from *assms(1)* **have** $[p = 3] \pmod{4}$
by (*simp add: unique-euclidean-semiring-class.cong-def*)
then have $[p + 1 = 3 + 1] \pmod{4}$
using *cong-add-rcancel-nat* **by** *blast*
then have $[p + 1 = 0] \pmod{4}$
by (*simp add: unique-euclidean-semiring-class.cong-def*)
then have $4 \text{ dvd } p + 1$
using *cong-0-iff* **by** *blast*
then have $x \wedge (2 * ((p + 1) \text{ div } 4)) = x \wedge ((p + 1) \text{ div } 2)$
by *auto*
from *x-mod-1* **this have** *x-mod-2*: $(x \wedge (2 \wedge (0 + 1))) \wedge (((p + 1) \text{ div } 4) \wedge (0 + 1)) = x \wedge ((p + 1) \text{ div } 2)$
by *auto*

from *x-mod-2* *assms(1)* *assms(3)* **have** *p-greater-3*: $p \geq 3$ **by** *auto*
from *assms(3)* **have** *odd-p*: *odd p*
by (*simp add: assms(1) even-even-mod-4-iff*)
from *odd-p p-greater-3* **have** *p-add-1[symmetric]*: $(p + 1) \text{ div } 2 = 1 + (p - 1) \text{ div } 2$
by *linarith*
from *this x-mod-2 odd-p p-greater-3* **have** $(x \wedge (2 \wedge (0 + 1))) \wedge (((p + 1) \text{ div } 4) \wedge (0 + 1)) = x \wedge (1 + (p - 1) \text{ div } 2)$
by *auto*
then have $(x \wedge (2 \wedge (0 + 1))) \wedge (((p + 1) \text{ div } 4) \wedge (0 + 1)) = x * x \wedge ((p - 1) \text{ div } 2)$ **by** *auto*
then have *x-x-p*: $[(x \wedge (2 \wedge (0 + 1))) \wedge (((p + 1) \text{ div } 4) \wedge (0 + 1)) = x * x \wedge ((p - 1) \text{ div } 2)] \pmod{p}$
by *auto*
from *assms(5)* **have** *coprime-xp*: *coprime x p* **by** *auto*
then have *coprime-px*: *coprime p x*
using *coprime-commute* **by** *auto*
from *assms(6)* **have** $\exists r. [x = r * r] \pmod{p}$
using *cong-modulus-mult-nat* **by** *blast*
then have *x-is-squared*: $\exists r. [r^2 = x] \pmod{p}$
by (*simp add: cong-sym-eq power2-eq-square*)

from *eulers-criterion assms(3) odd-p coprime-px x-is-squared* **have** *euler-thm*:
 $[x \wedge ((p - 1) \text{ div } 2) = 1] \pmod{p}$


```

    by auto
    have  $[x = x] \pmod{p}$  by auto
    from this euler-thm have  $[x * x \wedge ((p - 1) \text{ div } 2) = x * 1] \pmod{p}$  using
cong-mult by blast
    then have  $[x * x \wedge ((p - 1) \text{ div } 2) = x] \pmod{p}$  by auto
    from this x-x-p show ?case using cong-trans by auto
next
  case (Suc t)
  assume  $[(x \wedge 2 \wedge (t + 1)) \wedge ((p + 1) \text{ div } 4) \wedge (t + 1) = x] \pmod{p}$ 

  then have  $[(x \wedge 2 \wedge (t + 1)) \wedge ((p + 1) \text{ div } 4) \wedge (t + 1) \wedge ((p + 1) \text{ div } 4) = x \wedge ((p + 1) \text{ div } 4)] \pmod{p}$ 
  using cong-pow by blast

  from assms(1) have  $[p = 3] \pmod{4}$ 
  by (simp add: unique-euclidean-semiring-class.cong-def)
  then have  $[p + 1 = 3 + 1] \pmod{4}$ 
  using cong-add-rcancel-nat by blast
  then have  $[p + 1 = 0] \pmod{4}$ 
  by (simp add: unique-euclidean-semiring-class.cong-def)
  then have p-dvd-by-4:  $4 \text{ dvd } p + 1$ 
  using cong-0-iff by blast

  from assms(1) assms(3) have p-greater-3:  $p \geq 3$  by auto
  from assms(3) have odd-p: odd p
  by (simp add: assms(1) even-even-mod-4-iff)
  from odd-p p-greater-3 have p-add-1[symmetric]:  $(p + 1) \text{ div } 2 = 1 + (p - 1) \text{ div } 2$ 
  by linarith
  have x-refl:  $[x = x] \pmod{p}$  by auto

  from assms(5) have coprime-xp: coprime x p by auto
  then have coprime-p-x: coprime p x using coprime-commute by blast
  from assms(6) have  $\exists r. [x = r * r] \pmod{p}$ 
  using cong-modulus-mult-nat by blast
  then have x-is-squared:  $\exists r. [r^2 = x] \pmod{p}$ 
  by (auto simp add: cong-sym-eq power2-eq-square)

  from eulers-criterion assms(3) odd-p coprime-p-x x-is-squared have euler-thm:
 $[x \wedge ((p - 1) \text{ div } 2) = 1] \pmod{p}$ 
  by auto

  from this x-refl have  $[x * x \wedge ((p - 1) \text{ div } 2) = x * 1] \pmod{p}$  using cong-mult
by blast
  then have cong-xx:  $[x * x \wedge ((p - 1) \text{ div } 2) = x] \pmod{p}$  by auto
  have  $x * x \wedge ((p - 1) \text{ div } 2) = x \wedge (1 + (p - 1) \text{ div } 2)$  using powr-add by
auto
  then have  $x * x \wedge ((p - 1) \text{ div } 2) = x \wedge ((p + 1) \text{ div } 2)$  using p-add-1 by
algebra

```

```

from cong-xx this have  $[x \wedge ((p + 1) \text{ div } 2) = x] \pmod{p}$  by auto

from this Suc have cong-1:  $(((x \wedge 2 \wedge (t + 1)) \wedge ((p + 1) \text{ div } 4) \wedge (t + 1)) \wedge ((p + 1) \text{ div } 2) = x] \pmod{p}$ 
using cong-pow cong-trans by blast

have help-1:  $((p + 1) \text{ div } 4) \wedge (t + 2)) * 2 = 2 * ((p + 1) \text{ div } 4) \wedge (t + 2))$ 
by auto
have help-2:  $((p + 1) \text{ div } 4) \wedge (t + 1)) * ((p + 1) \text{ div } 4) * 2 = ((p + 1) \text{ div } 4) \wedge (t + 2)) * 2$  by auto

from p-dvd-by-4 have div-div[symmetric]:  $((p + 1) \text{ div } 4) * 2 = (p + 1) \text{ div } 2$ 
by fastforce

have eq-1:  $((x \wedge 2 \wedge (t + 1)) \wedge ((p + 1) \text{ div } 4) \wedge (t + 1)) \wedge ((p + 1) \text{ div } 2)$ 
 $= ((x \wedge 2 \wedge (t + 1)) \wedge (((p + 1) \text{ div } 4) \wedge (t + 1)) * ((p + 1) \text{ div } 2)))$ 
by (simp add: power-mult)
also have eq-2:  $((x \wedge 2 \wedge (t + 1)) \wedge (((p + 1) \text{ div } 4) \wedge (t + 1)) * ((p + 1) \text{ div } 2)))$ 
 $= ((x \wedge 2 \wedge (t + 1)) \wedge (((p + 1) \text{ div } 4) \wedge (t + 1)) * ((p + 1) \text{ div } 4) * 2))$ 
using div-div by (auto simp add: more-arith-simps(11))
then also have eq-3:  $((x \wedge 2 \wedge (t + 1)) \wedge (((p + 1) \text{ div } 4) \wedge (t + 1)) * ((p + 1) \text{ div } 4) * 2))$ 
 $= ((x \wedge 2 \wedge (t + 1)) \wedge (((p + 1) \text{ div } 4) \wedge (t + 2)) * 2))$ 
using help-2 by algebra

also have eq-4:  $((x \wedge 2 \wedge (t + 1)) \wedge (((p + 1) \text{ div } 4) \wedge (t + 2)) * 2))$ 
 $= ((x \wedge 2 \wedge (t + 1)) \wedge (2 * ((p + 1) \text{ div } 4) \wedge (t + 2)))$  using help-1 by algebra

also have eq-5:  $((x \wedge 2 \wedge (t + 1)) \wedge (2 * ((p + 1) \text{ div } 4) \wedge (t + 2)))$ 
 $= (((x \wedge 2 \wedge (t + 1)) \wedge 2) \wedge ((p + 1) \text{ div } 4) \wedge (t + 2))$ 
using power-mult by auto
also have eq-6:  $((x \wedge 2 \wedge (t + 1)) \wedge 2) \wedge ((p + 1) \text{ div } 4) \wedge (t + 2))$ 
 $= ((x \wedge (2 \wedge (t + 1) * 2)) \wedge ((p + 1) \text{ div } 4) \wedge (t + 2))$ 
using power-mult[symmetric] by metis
also have eq-7:  $((x \wedge (2 \wedge (t + 1) * 2)) \wedge ((p + 1) \text{ div } 4) \wedge (t + 2)) =$ 
 $(x \wedge (2 \wedge (t + 2))) \wedge ((p + 1) \text{ div } 4) \wedge (t + 2)$  by auto
also have eq-8:  $(x \wedge (2 \wedge (t + 2))) \wedge ((p + 1) \text{ div } 4) \wedge (t + 2)$ 
 $= (x \wedge (2 \wedge (\text{Suc } t + 1))) \wedge ((p + 1) \text{ div } 4) \wedge (\text{Suc } t + 1)$  by auto
from cong-1 x-refl show ?case
using calculation by auto
qed

```

lemma *exp-mod-prime-p*:

assumes *prime p coprime a p*
shows $[a \wedge b = a \wedge (b \bmod (p - 1))] \pmod{p}$
proof –
from *euler-theorem assms(2)* **have** $[a \wedge (\text{totient } p) = 1] \pmod{p}$ **by** *auto*
from *this totient-prime assms(1)* **have** *a-mod-1*: $[a \wedge (p - 1) = 1] \pmod{p}$ **by** *auto*
from *mod-div-decomp* **have** *b-decomp*: $b = (b \text{ div } (p - 1)) * (p - 1) + (b \bmod (p - 1))$ **by** *auto*
then have *a-exp-decomp*: $[a \wedge b = a \wedge ((b \text{ div } (p - 1)) * (p - 1) + (b \bmod (p - 1)))] \pmod{p}$ **by** *auto*

have *a-Dec1*: $a \wedge ((b \text{ div } (p - 1)) * (p - 1) + (b \bmod (p - 1))) = a \wedge ((b \text{ div } (p - 1)) * (p - 1)) * a \wedge (b \bmod (p - 1))$
using *power-add* **by** *blast*
also have *a-Dec2*: $a \wedge ((b \text{ div } (p - 1)) * (p - 1)) * a \wedge (b \bmod (p - 1)) = a \wedge ((p - 1) * (b \text{ div } (p - 1))) * a \wedge (b \bmod (p - 1))$
by *(simp add: Groups.mult-ac(2))*
also have $a \wedge ((p - 1) * (b \text{ div } (p - 1))) = (a \wedge (p - 1)) \wedge (b \text{ div } (p - 1))$
using *power-mult* **by** *blast*
from *this a-Dec1 a-Dec2* **have** $a \wedge ((b \text{ div } (p - 1)) * (p - 1) + (b \bmod (p - 1))) = (a \wedge (p - 1)) \wedge (b \text{ div } (p - 1)) * a \wedge (b \bmod (p - 1))$
by *auto*

from *a-exp-decomp this* **have** *a-b-cong*: $[a \wedge b = (a \wedge (p - 1)) \wedge (b \text{ div } (p - 1)) * a \wedge (b \bmod (p - 1))] \pmod{p}$ **by** *auto*
from *a-mod-1* **have** *a-exp-mod-1*: $[(a \wedge (p - 1)) \wedge (b \text{ div } (p - 1)) = 1] \pmod{p}$ **using** *cong-pow* **by** *fastforce*
have $[a \wedge (b \bmod (p - 1)) = a \wedge (b \bmod (p - 1))] \pmod{p}$ **by** *auto*

from *this a-exp-mod-1* **have** $[(a \wedge (p - 1)) \wedge (b \text{ div } (p - 1)) * a \wedge (b \bmod (p - 1)) = 1 \wedge (b \text{ div } (p - 1)) * a \wedge (b \bmod (p - 1))] \pmod{p}$
using *cong-scalar-right* **by** *fastforce*

from *this a-b-cong* **have** $[a \wedge b = 1 \wedge (b \text{ div } (p - 1)) * a \wedge (b \bmod (p - 1))] \pmod{p}$ **using** *cong-trans* **by** *fast*
then show *?thesis* **by** *auto*
qed

lemma *x-exp-sym*:

assumes $p \bmod 4 = 3$ $q \bmod 4 = 3$
prime (p::nat) *prime q*
*coprime x (p * q)* $\exists r. [x = r * r] \pmod{p * q}$
shows $[(x \wedge (2 \wedge (t + 1))) \wedge (((q + 1) \text{ div } 4) \wedge (t + 1)) = x] \pmod{q}$
by *(metis Groups.mult-ac(2) assms(1) assms(2) assms(3) assms(4) assms(5) assms(6) x-exp)*

lemma *same-x-0*:

assumes

$p \bmod 4 = 3$ $q \bmod 4 = 3$
 $\text{prime } (p::\text{nat})$ $\text{prime } q$
 $r < p * q$ $\text{coprime } r (p * q)$
 $p \neq q$

shows

$(\text{let } x = r \wedge (2 \wedge (t + 2)) \bmod (p * q) \text{ in}$
 $\text{let } u-p = x \wedge (\text{calculate-exponent } t \ p) \bmod p \text{ in}$
 $\text{let } u-q = x \wedge (\text{calculate-exponent } t \ q) \bmod q \text{ in}$
 $\text{let } ((r-p, r-q), -) = \text{euclid-ext } p \ q \text{ in}$
 $\text{nat } ((\text{int } u-q * r-p * \text{int } p + \text{int } u-p * r-q * \text{int } q) \bmod \text{int } (p * q))) = (r \wedge 2)$
 $\bmod (p * q)$

proof –

let $?xt = r \wedge (2 \wedge (t + 2)) \bmod (p * q)$
let $?x0 = (r * r) \bmod p * q$
let $?up = ?xt \wedge (\text{calculate-exponent } t \ p) \bmod p$
let $?uq = ?xt \wedge (\text{calculate-exponent } t \ q) \bmod q$

have $?xt = r \wedge (2 * 2 \wedge (t + 1)) \bmod (p * q)$ **by** *auto*

also have $r \wedge (2 * 2 \wedge (t + 1)) \bmod (p * q) = (r \wedge 2) \wedge (2 \wedge (t + 1)) \bmod (p * q)$

by *(simp add: power-mult)*

then have $x-t\text{-def}: ?xt = (r \wedge 2) \wedge (2 \wedge (t + 1)) \bmod (p * q)$ **by** *auto*

have $[?up = ?xt \wedge (\text{calculate-exponent } t \ p)] \bmod p$ **by** *auto*

then have $u-p-1: [?up = ?xt \wedge (((\text{Suc } p) \text{ div } 4) \wedge (\text{Suc } t) \bmod (p-1))] \bmod p$
by *auto*

from $x-t\text{-def}$ **assms** **have** $[?xt = (r \wedge 2) \wedge (2 \wedge (t + 1))] \bmod p$

using *cong-refl mod-mult-cong-right*

by *metis*

from $u-p-1$ **this** **have** $\text{cong-up}: [?up = ((r \wedge 2) \wedge (2 \wedge (t + 1))) \wedge (((\text{Suc } p) \text{ div } 4) \wedge (\text{Suc } t) \bmod (p-1))] \bmod p$

using *cong-pow* **by** *fastforce*

from $\text{assms}(6)$ **have** $\text{coprime } r \ p$ **by** *auto*

then have $\text{coprime-xp}: \text{coprime } ((r \wedge 2) \wedge (2 \wedge (t + 1))) \ p$ **by** *auto*

from $\text{assms}(6)$ **have** $\text{coprime-r-sq}: \text{coprime } (r \wedge 2) \ (p * q)$ **by** *auto*

from coprime-xp $\text{assms}(3)$

have $[((r \wedge 2) \wedge (2 \wedge (t + 1))) \wedge (((\text{Suc } p) \text{ div } 4) \wedge (\text{Suc } t))] \bmod p$
 $= ((r \wedge 2) \wedge (2 \wedge (t + 1))) \wedge (((\text{Suc } p) \text{ div } 4) \wedge (\text{Suc } t) \bmod (p-1))] \bmod p$

```

    using exp-mod-prime-p by auto
    then have mod-exp-elim:  $[(r \wedge 2) \wedge (2 \wedge (t + 1))] \wedge (((\text{Suc } p) \text{ div } 4) \wedge (\text{Suc } t) \bmod (p-1))$ 
      =  $[(r \wedge 2) \wedge (2 \wedge (t + 1))] \wedge (((\text{Suc } p) \text{ div } 4) \wedge (\text{Suc } t)) \bmod p$ 
    using cong-sym by auto

    have [ $?up = ?up$ ] (mod p) by auto
    from cong-up mod-exp-elim have up-eq: [ $?up = (r \wedge 2) \wedge (2 \wedge (t + 1)) \wedge (((\text{Suc } p) \text{ div } 4) \wedge (\text{Suc } t)) \bmod p$ ]
    using cong-trans by fast

    from assms(1) assms(2) assms(3) assms(4) coprime-r-sq have x-t-to-x-zero:
 $[(r \wedge 2) \wedge (2 \wedge (t + 1))] \wedge (((\text{Suc } p) \text{ div } 4) \wedge (\text{Suc } t)) = r \wedge 2 \bmod p$ 
    using x-exp cong-refl power2-eq-square semiring-norm(174)
    by (metis)

    from up-eq x-t-to-x-zero have up-cong-x-mod-p: [ $?up = r \wedge 2$ ] (mod p)
    using cong-trans by fast

    have [ $?uq = ?xt \wedge (\text{calculate-exponent } t \ q)$ ] (mod q) by auto
    then have u-q-1: [ $?uq = ?xt \wedge (((\text{Suc } q) \text{ div } 4) \wedge (\text{Suc } t) \bmod (q-1))$ ] (mod q)
    by auto

    from x-t-def have ?xt =  $(r \wedge 2) \wedge (2 \wedge (t + 1)) \bmod (q * p)$ 
    by (simp add: mult.commute)
    from this assms have [ $?xt = (r \wedge 2) \wedge (2 \wedge (t + 1))$ ] (mod q)
    using cong-refl mod-mult-cong-right
    by metis

    from u-q-1 this have cong-up: [ $?uq = ((r \wedge 2) \wedge (2 \wedge (t + 1))) \wedge (((\text{Suc } q) \text{ div } 4) \wedge (\text{Suc } t) \bmod (q-1))$ ] (mod q)
    using cong-pow by fastforce

    from assms(6) have coprime r q by auto
    then have coprime-xq: coprime  $((r \wedge 2) \wedge (2 \wedge (t + 1))) \ q$  by auto
    from assms(6) have coprime-r-sq-rev: coprime  $(r \wedge 2) \ (q * p)$  by auto

    from coprime-xq assms(4)
    have  $[(r \wedge 2) \wedge (2 \wedge (t + 1))] \wedge (((\text{Suc } q) \text{ div } 4) \wedge (\text{Suc } t))$ 
      =  $((r \wedge 2) \wedge (2 \wedge (t + 1))) \wedge (((\text{Suc } q) \text{ div } 4) \wedge (\text{Suc } t) \bmod (q-1)) \bmod q$ 
    using exp-mod-prime-p by auto

    then have mod-exp-elim:  $[(r \wedge 2) \wedge (2 \wedge (t + 1))] \wedge (((\text{Suc } q) \text{ div } 4) \wedge (\text{Suc } t) \bmod (q-1))$ 
      =  $((r \wedge 2) \wedge (2 \wedge (t + 1))) \wedge (((\text{Suc } q) \text{ div } 4) \wedge (\text{Suc } t)) \bmod q$ 
    using cong-sym by auto

```

```

have [ $?up = ?up$ ] (mod  $q$ ) by auto
from cong-up mod-exp-elim have uq-eq: [ $?uq = ((r \wedge 2) \wedge (2 \wedge (t + 1))) \wedge$   

 $((\text{Suc } q) \text{ div } 4) \wedge (\text{Suc } t))$ ] (mod  $q$ )
using cong-trans by fast

from assms(1) assms(2) assms(3) assms(4) coprime-r-sq
have x-t-to-x-zero: [ $((r \wedge 2) \wedge (2 \wedge (t + 1))) \wedge ((\text{Suc } q) \text{ div } 4) \wedge (\text{Suc } t)) = r$   

 $\wedge 2$ ] (mod  $q$ )
using x-exp-sym cong-refl power2-eq-square semiring-norm(174)
by (metis)

from uq-eq x-t-to-x-zero have up-cong-x-mod-q: [ $?uq = r \wedge 2$ ] (mod  $q$ )
using cong-trans by fast

have  $\exists r-p \ r-q \ c. (((r-p, r-q), c) = \text{euclid-ext } p \ q)$ 
using surj-pair
by (metis)
then obtain  $r-p \ r-q \ c$  where r-obtain:  $((r-p, r-q), c) = \text{euclid-ext } p \ q$  by blast
hence c-def:  $c = \text{gcd } p \ q$  by auto
from assms(3) assms(4) assms(7) have coprime-pq: coprime  $p \ q$ 
using Primes.primes-coprime-int by auto
from this c-def have  $c = 1$  by auto

from coprime-pq have gcd-pq:  $\text{gcd } p \ q = 1$  by auto

from r-obtain have bezout-coefficients  $p \ q = (r-p, r-q)$  by auto
from gcd-pq this have  $r-p * p + r-q * q = 1$ 
using bezout-coefficients
by fastforce
hence  $(r \wedge 2) * (r-p * p + r-q * q) = (r \wedge 2) * 1$  by auto
hence final-eq:  $((r \wedge 2) * r-p * p + (r \wedge 2) * r-q * q) = (r \wedge 2)$ 
by (simp add: int-distrib(2))

have p-times-mod-0:  $[(r \wedge 2) * r-p * p = 0]$  (mod  $p$ )
using cong-mult-self-right by blast
from final-eq
have  $[(r \wedge 2) * r-p * p + (r \wedge 2) * r-q * q = (r \wedge 2)]$  (mod  $p$ ) by auto
then have cong-1:  $[(r \wedge 2) = ((r \wedge 2) * r-p * p + (r \wedge 2) * r-q * q)]$  (mod  $p$ )
using cong-sym by auto

have  $[(r \wedge 2) * r-q * q = (r \wedge 2) * r-q * q]$  (mod  $p$ ) by auto

from this p-times-mod-0
have cong-2:  $[(r \wedge 2) * r-p * p + (r \wedge 2) * r-q * q = 0 + (r \wedge 2) * r-q * q]$   

(mod  $p$ )

```

```

    using cong-add by fastforce
  from cong-1 cong-2
  have  $[(r \wedge 2) = 0 + (r \wedge 2) * r-q * q] \pmod p$ 
    using cong-trans
    by fastforce
  hence p-cong:  $[(r \wedge 2) = (r \wedge 2) * r-q * q] \pmod p$  by auto
  from up-cong-x-mod-p have up-cong:  $[(r \wedge 2) = ?up] \pmod p$  using cong-sym
by auto
  from p-cong up-cong
  have p-cong-u:  $[(r \wedge 2) = ?up * r-q * q] \pmod p$ 
    using cong-int-iff cong-scalar-right cong-trans
    by (smt (verit, ccfv-threshold) )

  have p-times-mod-0:  $[?uq * r-p * p = 0] \pmod p$ 
    using cong-mult-self-right by blast
  from p-cong-u have  $[?up * r-q * q = (r \wedge 2)] \pmod p$  using cong-sym-eq by
blast
  from this p-times-mod-0 have p-cong-u2:  $[(?uq * r-p * p + ?up * r-q * q) = (r \wedge 2)] \pmod p$ 
    using cong-add by fastforce

  have q-times-mod-0:  $[(r \wedge 2) * r-q * q = 0] \pmod q$ 
    using cong-mult-self-right by blast
  from final-eq
  have  $[(r \wedge 2) * r-p * p + (r \wedge 2) * r-q * q = (r \wedge 2)] \pmod q$  by auto
  then have cong-1:  $[(r \wedge 2) = ((r \wedge 2) * r-p * p + (r \wedge 2) * r-q * q)] \pmod q$ 
    using cong-sym by auto

  have  $[(r \wedge 2) * r-p * p = (r \wedge 2) * r-p * p] \pmod q$  by auto

  from this q-times-mod-0
  have cong-2:  $[(r \wedge 2) * r-p * p + (r \wedge 2) * r-q * q = (r \wedge 2) * r-p * p + 0] \pmod q$ 
    using cong-add by fast
  from cong-1 cong-2
  have  $[(r \wedge 2) = 0 + (r \wedge 2) * r-p * p] \pmod q$ 
    using cong-trans
    by fastforce
  hence q-cong:  $[(r \wedge 2) = (r \wedge 2) * r-p * p] \pmod q$  by auto

  from up-cong-x-mod-q have uq-cong:  $[(r \wedge 2) = ?uq] \pmod q$  using cong-sym
by auto
  from q-cong uq-cong
  have q-cong-u:  $[(r \wedge 2) = ?uq * r-p * p] \pmod q$ 
    using cong-int-iff cong-scalar-right cong-trans
    by (smt (verit, ccfv-threshold) )

```

```

have q-times-mod-0: [ $?up * r - q * q = 0$ ] (mod  $q$ )
  using cong-mult-self-right by blast
from q-cong-u have [ $?uq * r - p * p = (r \wedge 2)$ ] (mod  $q$ ) using cong-sym-eq by
blast
from this q-times-mod-0 have q-cong-u2: [ $(?uq * r - p * p + ?up * r - q * q) = (r \wedge 2)$ ] (mod  $q$ )
  using cong-add by fastforce

```

```

from p-cong-u2 q-cong-u2 coprime-pq
have pq-cong: [ $(?uq * r - p * p + ?up * r - q * q) = (r \wedge 2)$ ] (mod  $(p * q)$ )
  using coprime-cong-mult by fastforce
then have nat-cong:  $\text{nat } ((?uq * r - p * p + ?up * r - q * q) \text{ mod } (p * q)) = \text{nat } (r \wedge 2) \text{ mod } (p * q)$ 
  by (metis nat-int of-nat-mod unique-euclidean-semiring-class.cong-def)

```

```

have  $r \wedge 2 = \text{nat } (\text{int } (r \wedge 2))$  by presburger
from this nat-cong have nat-eq:  $\text{nat } ((?uq * r - p * p + ?up * r - q * q) \text{ mod } (p * q)) = r \wedge 2 \text{ mod } (p * q)$ 
  by auto

```

```

from r-obtain have
   $\text{nat } ((?uq * r - p * p + ?up * r - q * q) \text{ mod } (p * q)) =$ 
    ( $\text{let } ((r - p, r - q), -) = \text{euclid-ext } p \ q \text{ in}$ 
       $\text{nat } ((?uq * r - p * p + ?up * r - q * q) \text{ mod } (p * q))$ )
using old.prod.case
by (metis (no-types, lifting))

```

```

from this nat-eq
have let-eq: ( $\text{let } ((r - p, r - q), -) = \text{euclid-ext } p \ q \text{ in}$ 
   $\text{nat } ((?uq * r - p * p + ?up * r - q * q) \text{ mod } (p * q))$ 
   $= r \wedge 2 \text{ mod } (p * q)$ ) by algebra

```

```

have unwrap: ( $\text{let } x = r \wedge (2 \wedge (t + 2)) \text{ mod } (p * q) \text{ in}$ 
   $\text{let } u - p = x \wedge (\text{calculate-exponent } t \ p) \text{ mod } p \text{ in}$ 
   $\text{let } u - q = x \wedge (\text{calculate-exponent } t \ q) \text{ mod } q \text{ in}$ 
   $\text{let } ((r - p, r - q), -) = \text{euclid-ext } p \ q \text{ in}$ 
   $\text{nat } ((u - q * r - p * p + u - p * r - q * q) \text{ mod } (p * q)) =$ 
  ( $\text{let } ((r - p, r - q), -) = \text{euclid-ext } p \ q \text{ in}$ 
     $\text{nat } ((?uq * r - p * p + ?up * r - q * q) \text{ mod } (p * q))$ )
  by metis

```

```

have ( $\text{let } ((r - p, r - q), -) = \text{euclid-ext-aux } 1 \ 0 \ 0 \ 1 \ (\text{int } p) \ (\text{int } q)$ )

```



```

    in nat ((int ((r ^ 2 ^ (t + 2) mod (p * q)) ^ calculate-exponent t q mod q) *
r-p * int p +
    int ((r ^ 2 ^ (t + 2) mod (p * q)) ^ calculate-exponent t p mod p) *
r-q * int q) mod
    int (p * q))) =
    (let ((r-p, r-q), -) = euclid-ext-aux 1 0 0 1 (int p) (int q)
    in nat ((int ((r ^ 2 ^ (t + 2) mod (p * q)) ^ calculate-exponent t q mod q) *
r-p * int p +
    int ((r ^ 2 ^ (t + 2) mod (p * q)) ^ calculate-exponent t p mod p) *
r-q * int q) mod
    int (p * q))) by auto

from let-eq unwrap this have (let x = r ^ (2 ^ (t + 2)) mod (p * q) in
    let u-p = x ^ (calculate-exponent t p) mod p in
    let u-q = x ^ (calculate-exponent t q) mod q in
    let ((r-p, r-q), -) = euclid-ext p q in
    nat ((u-q * r-p * p + u-p * r-q * q) mod (p * q))) = r ^ 2 mod (p * q) by
algebra
from this show ?thesis by fastforce
qed

```

lemma *h-not-smaller*: $h \geq \text{length } (\text{nat-to-words } (\text{Suc } 0) (p \bmod 2^h))$
by (simp add: nat-bnd-word-len)

lemma *length-eq*: $\text{length } (\text{nat-to-bitstring } h (p \bmod 2^h)) = h$
by (auto simp add: nat-to-bitstring-def nat-to-words-len-def h-not-smaller)

Here, we show that if we find the x_0 , then we can recover the plaintext message from encrypted message.

lemma *enc-loop-alt-correctness*:
assumes $x > 0 \ n > 0 \ h > 0 \ x < n$
shows $\llbracket \forall l \in \text{set } m. \text{length } l \leq h; c = \text{fst } (\text{enc-loop-alt } (n, h, m) x) \rrbracket \implies$
 $\text{fst } (\text{enc-loop-alt } (n, h, c) x) = m$

proof (induction m arbitrary: c x)
case Nil
then show ?case **by** auto
next
let ?f = nat-to-bitstring h
case (Cons a m)
assume $\forall l \in \text{set } (a \# m). \text{length } l \leq h$
assume $c = \text{fst } (\text{enc-loop-alt } (n, h, a \# m) x)$

then have $c = \text{enc-loop-func-list } (?f, n, h, a \# m) x$ **by** auto
then have c-def: $c = (\text{let } x-i = x * x \bmod n \text{ in}$
 $\text{let } p-i = ?f (x-i \bmod 2^h) \text{ in}$
 $\text{let } c-i = a [\oplus] p-i \text{ in}$
 $c-i \# \text{enc-loop-func-list } (\text{nat-to-bitstring } h, n, h, m) x-i$

```

    ) by auto
  let ?a' = (a [⊕] ?f (x * x mod n mod 2^h))

  from c-def have c-part: c = ?a' # enc-loop-func-list (nat-to-bitstring h, n, h,
m) (x * x mod n) by metis
  then have ∃ c'. c = ?a' # c' by auto
  then obtain c' where c-obtain: c = ?a' # c' by blast
  from this c-part have c' = enc-loop-func-list (nat-to-bitstring h, n, h, m) (x *
x mod n) by auto
  hence fst (enc-loop-alt (n, h, c') (x * x mod n)) = m using Cons by auto
  hence a-with-m: a # fst (enc-loop-alt (n, h, c') (x * x mod n)) = a # m by
auto

  have a-xor-bot: ?a' [⊕] ?f (x * x mod n mod 2^h) = a [⊕] (replicate (length (?f
(x * x mod n mod 2^h))) bot)
  by auto

  have length a ≤ h using Cons(2) by auto
  then have a [⊕] (replicate (length (?f (x * x mod n mod 2^h))) bot) = a
  using xor-replicate-bot-right length-eq by auto

  from this a-xor-bot have ?a' [⊕] ?f (x * x mod n mod 2^h) = a by auto

  from this a-with-m
  have ?a' [⊕] ?f (x * x mod n mod 2^h) # fst (enc-loop-alt (n, h, c') (x * x mod
n)) = a # m
  by auto

  hence (let x-i = x * x mod n in
    let p-i = ?f (x-i mod 2^h) in
    let c-i = ?a' [⊕] p-i in
    c-i # enc-loop-func-list (nat-to-bitstring h, n, h, c') x-i
  ) = a # m by auto
  hence enc-loop-func-list (nat-to-bitstring h, n, h, ?a' # c') x = a # m by auto

  then show ?case using c-obtain by auto
qed

lemma same-length:
  shows length (enc-loop-func-list (nat-to-bitstring h, p * q, h, m-l) x-0) = length
m-l
proof (induction m-l arbitrary: x-0)
  case Nil
  then show ?case by auto
next
  case (Cons a m-l)
  have enc-loop-open: length (enc-loop-func-list (nat-to-bitstring h, p * q, h, a #
m-l) x-0) =

```

```

length ((let x-i = x-0 * x-0 mod (p * q) in
  let p-i = nat-to-bitstring h (x-i mod 2^h) in
  let c-i = a [⊕] p-i in
  c-i # enc-loop-func-list (nat-to-bitstring h, p * q, h, m-l) x-i
)) by auto
then have ∃ c-i'. let x-i = x-0 * x-0 mod (p * q) in
  let p-i = nat-to-bitstring h (x-i mod 2^h) in
  c-i' = a [⊕] p-i by auto
then obtain c-i' where obtain-c-i: let x-i = x-0 * x-0 mod (p * q) in
  let p-i = nat-to-bitstring h (x-i mod 2^h) in
  c-i' = a [⊕] p-i by blast
from enc-loop-open this have enc-loop-open: length (enc-loop-func-list (nat-to-bitstring
h, p * q, h, a # m-l) x-0) =
  length (c-i' # enc-loop-func-list (nat-to-bitstring h, p * q, h, m-l) ((x-0 * x-0)
mod (p * q)))
by metis
then show ?case using Cons by auto
qed

```

lemma merge-split:

```

shows h > 0 ⇒ l = (split m h) ⇒ merge-list l = m
proof (induction m h arbitrary: l rule: split.induct)
  case (1 n)
    then show ?case by auto
  next
    case (2 v va)
      then show ?case by auto
  next
    case (3 x xs v)
      have split (x # xs) (Suc v) = (take (Suc v) (x # xs)) # (split (drop (Suc v) (x
# xs)) (Suc v))
      by (auto elim: split.elims)
      assume assms: 0 < Suc v l = split (x # xs) (Suc v)
      then have l-def: l = (take (Suc v) (x # xs)) # (split (drop (Suc v) (x # xs))
(Suc v))
      by auto
      then have ∃ a l-r. l = a # l-r by auto
      then obtain a l-r where a-l-r-obtain: l = a # l-r by auto
      from l-def this have a # l-r = (take (Suc v) (x # xs)) # (split (drop (Suc v)
(x # xs)) (Suc v))
      by auto
      then have a-def: a = (take (Suc v) (x # xs)) ∧ l-r = (split (drop (Suc v) (x
# xs)) (Suc v))
      by auto
      then have l-r-def: l-r = (split (drop (Suc v) (x # xs)) (Suc v)) by auto

from 3 have 0 < Suc v ⇒
  l-r = split (drop (Suc v) (x # xs)) (Suc v) ⇒ merge-list l-r = drop (Suc v)
(x # xs)

```

by auto
 then have $l-r = \text{split } (\text{drop } (\text{Suc } v) (x \# xs)) (\text{Suc } v)$
 $\implies \text{merge-list } l-r = \text{drop } (\text{Suc } v) (x \# xs)$ using *assms(1)* by auto
 then have *merge-l-r*: $\text{merge-list } l-r = \text{drop } (\text{Suc } v) (x \# xs)$ using *l-r-def* by auto
 auto

 have $\text{merge-list } (a \# l-r) = a @ \text{merge-list } l-r$ using *list.discI* by auto
 from *this merge-l-r*
 have $\text{merge-list } (a \# l-r) = a @ (\text{drop } (\text{Suc } v) (x \# xs))$ by auto
 then have $\text{merge-list } (a \# l-r) = (\text{take } (\text{Suc } v) (x \# xs)) @ (\text{drop } (\text{Suc } v) (x \# xs))$
 using *a-def* by auto

then show *?case*
 using *a-l-r-obtain* by auto
 qed

corollary *merge-split-in-place*:
 shows $h > 0 \implies \text{merge-list } (\text{split } m \ h) = m$
 using *merge-split* by auto

theorem *correctness*:

assumes

$$p \bmod 4 = 3 \quad q \bmod 4 = 3$$

prime p prime q

$$r < p * q \quad m \neq []$$

$$\text{coprime } r \ (p * q) \quad p \neq q$$

shows $\text{decrypt-alt } p \ q \ (\text{encrypt-alt } (p * q) \ m \ r) = m$

proof –

from *encrypt-alt-def* have *enc-def*: $\text{encrypt-alt } (p * q) \ m \ r = ($

$\text{let } h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ in

$\text{let } m-l = \text{split } m \ h$ in

$\text{let } (x-0 :: \text{nat}) = (r * r) \bmod (p * q)$ in

$\text{let } (c, x-t) = \text{enc-loop-alt } (p * q, h, m-l) \ x-0$ in

$\text{let } x-t' = x-t * x-t \bmod (p * q)$ in

$(c, x-t')$

by auto

then have $\text{fst } (\text{encrypt-alt } (p * q) \ m \ r)$

$= \text{fst } (\text{let } h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ in

$\text{let } m-l = \text{split } m \ h$ in

$\text{let } x-0 = (r * r) \bmod (p * q)$ in $\text{enc-loop-alt } (p * q, h, m-l) \ x-0)$

using *prod.exhaust prod.sel(1) prod.simps(2)*

by (*metis (mono-tags, lifting)*)

then have $\text{fst } (\text{encrypt-alt } (p * q) \ m \ r)$

$= (\text{let } h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ in

$\text{let } m-l = \text{split } m \ h$ in

$\text{let } x-0 = (r * r) \bmod (p * q)$ in $\text{fst } (\text{enc-loop-alt } (p * q, h, m-l) \ x-0))$

by *meson*

then have *enc-fst-simp*: $\text{fst } (\text{encrypt-alt } (p * q) \ m \ r)$
 $= (\text{let } h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor \text{ in}$
 $\text{let } m-l = \text{split } m \ h \text{ in}$
 $\text{let } x-0 = (r * r) \bmod (p * q) \text{ in } \text{enc-loop-func-list } (\text{nat-to-bitstring } h, p * q, h,$
 $m-l) \ x-0)$
by *auto*

have $\exists c. \text{fst } (\text{encrypt-alt } (p * q) \ m \ r) = c$ **by** *auto*
then obtain c **where** *obtain-c-opt1*: $\text{fst } (\text{encrypt-alt } (p * q) \ m \ r) = c$ **by** *blast*
then have *obtain-c-opt2*: $(\text{let } h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor \text{ in } \text{let } m-l = \text{split}$
 $m \ h \text{ in}$
 $\text{let } x-0 = (r * r) \bmod (p * q) \text{ in } \text{enc-loop-func-list } (\text{nat-to-bitstring } h, p * q, h,$
 $m-l) \ x-0) = c$
using *enc-fst-simp* **by** *auto*

from *enc-def* **have** $\text{snd } (\text{encrypt-alt } (p * q) \ m \ r)$
 $= \text{snd } (\text{let } h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor \text{ in}$
 $\text{let } m-l = \text{split } m \ h \text{ in}$
 $\text{let } x-0 = (r * r) \bmod (p * q) \text{ in}$
 $\text{let } (c, x-t) = \text{enc-loop-alt } (p * q, h, m-l) \ x-0 \text{ in}$
 $\text{let } x-t' = x-t * x-t \bmod (p * q) \text{ in}$
 $(c, x-t'))$
by *auto*

then have $\text{snd } (\text{encrypt-alt } (p * q) \ m \ r)$
 $= (\text{let } h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor \text{ in}$
 $\text{let } m-l = \text{split } m \ h \text{ in}$
 $\text{let } x-0 = (r * r) \bmod (p * q) \text{ in}$
 $\text{let } x-t = \text{snd } (\text{enc-loop-alt } (p * q, h, m-l) \ x-0) \text{ in}$
 $\text{let } x-t' = x-t * x-t \bmod (p * q) \text{ in}$
 $x-t')$
by (*smt* (*verit*, *ccfv-SIG*) *case-prod-unfold prod.sel(2)*)

then have $\text{snd } (\text{encrypt-alt } (p * q) \ m \ r)$
 $= (\text{let } h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor \text{ in}$
 $\text{let } m-l = \text{split } m \ h \text{ in}$
 $\text{let } x-0 = (r * r) \bmod (p * q) \text{ in}$
 $\text{let } x-t = \text{enc-loop-func-x } (\text{nat-to-bitstring } h, p * q, h, \text{length } m-l) \ x-0 \text{ in}$
 $\text{let } x-t' = x-t * x-t \bmod (p * q) \text{ in}$
 $x-t')$ **by** *auto*

then have $\text{snd } (\text{encrypt-alt } (p * q) \ m \ r)$
 $= (\text{let } h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor \text{ in}$
 $\text{let } m-l = \text{split } m \ h \text{ in}$
 $\text{let } x-0 = (r * r) \bmod (p * q) \text{ in}$
 $\text{let } x-t = x-0 \wedge (2 \wedge (\text{length } m-l)) \bmod (p * q) \text{ in}$
 $\text{let } x-t' = x-t * x-t \bmod (p * q) \text{ in}$

$x \cdot t$) **by** *auto*

then have *snd-open*: *snd* (*encrypt-alt* ($p * q$) m r)
 $=$ (*let* $h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ *in*
let $m-l = \text{split } m \text{ } h$ *in*
 $((((r * r) \bmod (p * q)) \wedge (2 \wedge (\text{length } m-l)) \bmod (p * q))$
 $* (((r * r) \bmod (p * q)) \wedge (2 \wedge (\text{length } m-l)) \bmod (p * q)))) \bmod (p * q)$
by *metis*

have [*let* $h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ *in*
let $m-l = \text{split } m \text{ } h$ *in*
 $((((r * r) \bmod (p * q)) \wedge (2 \wedge (\text{length } m-l)) \bmod (p * q))$
 $* (((r * r) \bmod (p * q)) \wedge (2 \wedge (\text{length } m-l)) \bmod (p * q)))) \bmod (p * q)$
 $=$ (*let* $h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ *in*
let $m-l = \text{split } m \text{ } h$ *in*
 $(r * r) \wedge (2 \wedge (\text{length } m-l))$
 $* (r * r) \wedge (2 \wedge (\text{length } m-l))]$ (*mod* ($p * q$))
by (*metis* (*no-types*, *opaque-lifting*) *cong-mod-left* *cong-refl* *power2-eq-square* *power-mod*)

from *this* *snd-open* **have** *snd* (*encrypt-alt* ($p * q$) m r)
 $=$ (*let* $h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ *in*
let $m-l = \text{split } m \text{ } h$ *in*
 $(r * r) \wedge (2 \wedge (\text{length } m-l))$
 $* (r * r) \wedge (2 \wedge (\text{length } m-l))$) *mod* ($p * q$)
by (*metis* *power2-eq-square* *power-mod*)

then have *snd* (*encrypt-alt* ($p * q$) m r)
 $=$ (*let* $h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ *in*
let $m-l = \text{split } m \text{ } h$ *in*
 $((r * r) \wedge (2 \wedge (\text{length } m-l) + 2 \wedge (\text{length } m-l))))$
mod ($p * q$) **using** *power-add[symmetric]*
by *metis*

then have *snd* (*encrypt-alt* ($p * q$) m r)
 $=$ (*let* $h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ *in*
let $m-l = \text{split } m \text{ } h$ *in*
 $((r * r) \wedge (2 * 2 \wedge (\text{length } m-l))))$
mod ($p * q$) **using** *mult-2[symmetric]*
by *metis*

then have *snd-eq*: *snd* (*encrypt-alt* ($p * q$) m r)
 $=$ (*let* $h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ *in*
let $m-l = \text{split } m \text{ } h$ *in*
 $(r * r) \wedge (2 \wedge (\text{Suc } (\text{length } m-l))))$
mod ($p * q$) **by** *auto*

then have $\exists h. h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$ **by** *auto*

```

then obtain  $h$  where  $h$ -obtain:  $h = \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor$  by blast

have  $\exists x. x = \text{snd } (\text{encrypt-alt } (p * q) \ m \ r)$  by auto
then obtain  $x$  where  $\text{obtain-x-opt1}: x = \text{snd } (\text{encrypt-alt } (p * q) \ m \ r)$  by blast

from  $h$ -obtain have  $\text{snd } (\text{encrypt-alt } (p * q) \ m \ r) = (r * r) \wedge (2 \wedge (\text{Suc } (\text{length } (\text{split } m \ h)))) \bmod (p * q)$ 
using snd-eq by presburger
then have  $\text{snd } (\text{encrypt-alt } (p * q) \ m \ r) = (r \wedge 2) \wedge (2 \wedge (\text{Suc } (\text{length } (\text{split } m \ h)))) \bmod (p * q)$ 
using power2-eq-square[symmetric] by metis
then have  $\text{snd } (\text{encrypt-alt } (p * q) \ m \ r) = r \wedge (2 \wedge ((\text{length } (\text{split } m \ h)) + 2)) \bmod (p * q)$ 
by (auto simp add: power-mult)
then have  $\text{obtain-x-opt2}: x = r \wedge (2 \wedge ((\text{length } (\text{split } m \ h)) + 2)) \bmod (p * q)$ 
using obtain-x-opt1 by auto

from  $\text{obtain-c-opt2}$   $h$ -obtain
have  $\text{obtain-c-opt3}: c = \text{enc-loop-func-list } (\text{nat-to-bitstring } h, p * q, h, (\text{split } m \ h)) (r * r \bmod (p * q))$ 
by metis

from assms(1) assms(2) assms(3) assms(4) assms(5) assms(7) assms(8)
same-x-0
have  $x\text{-0-eq}: (\text{let } x = r \wedge 2 \wedge (\text{length } (\text{split } m \ h) + 2) \bmod (p * q);$ 
 $u\text{-}p = x \wedge \text{calculate-exponent } (\text{length } (\text{split } m \ h)) \ p \bmod p;$ 
 $u\text{-}q = x \wedge \text{calculate-exponent } (\text{length } (\text{split } m \ h)) \ q \bmod q;$ 
 $((r\text{-}p, r\text{-}q), -) = \text{euclid-ext-aux } 1 \ 0 \ 0 \ 1 \ (\text{int } p) \ (\text{int } q)$ 
in
 $\text{nat } ((\text{int } u\text{-}q * r\text{-}p * \text{int } p + \text{int } u\text{-}p * r\text{-}q * \text{int } q) \bmod \text{int } (p * q))) =$ 
 $r \wedge 2 \bmod (p * q)$  by blast

from same-length  $\text{obtain-c-opt2}$ 
have  $\text{length } (\text{enc-loop-func-list } (\text{nat-to-bitstring } h, p * q, h, (\text{split } m \ h)) (r * r \bmod (p * q)))$ 
 $= \text{length } (\text{split } m \ h)$ 
by auto
from this  $\text{obtain-c-opt3}$  have  $\text{length-eq}: \text{length } c = \text{length } (\text{split } m \ h)$  by auto
from  $\text{length-eq}$   $\text{obtain-x-opt2}$  have  $\text{obtain-x-opt3}: x = r \wedge 2 \wedge (\text{length } c + 2) \bmod (p * q)$ 
by auto

from  $\text{length-eq}$   $x\text{-0-eq}$  have  $x\text{-0-eqc}: (\text{let } x = r \wedge 2 \wedge (\text{length } c + 2) \bmod (p * q);$ 
 $u\text{-}p = x \wedge \text{calculate-exponent } (\text{length } c) \ p \bmod p;$ 
 $u\text{-}q = x \wedge \text{calculate-exponent } (\text{length } c) \ q \bmod q;$ 
 $((r\text{-}p, r\text{-}q), -) = \text{euclid-ext } (\text{int } p) \ (\text{int } q)$ 
in
 $\text{nat } ((\text{int } u\text{-}q * r\text{-}p * \text{int } p + \text{int } u\text{-}p * r\text{-}q * \text{int } q) \bmod \text{int } (p * q))) =$ 

```

$r \wedge 2 \bmod (p * q)$ **by algebra**

then have (let $u-p = x \wedge \text{calculate-exponent } (\text{length } c) \text{ } p \bmod p$;
 $u-q = x \wedge \text{calculate-exponent } (\text{length } c) \text{ } q \bmod q$;
 $((r-p, r-q), -) = \text{euclid-ext } (\text{int } p) (\text{int } q)$
in
 $\text{nat } ((\text{int } u-q * r-p * \text{int } p + \text{int } u-p * r-q * \text{int } q) \bmod \text{int } (p * q))) =$
 $r \wedge 2 \bmod (p * q)$ **using obtain-x-opt3 by meson**

then have (let $((r-p, r-q), -) = \text{euclid-ext } (\text{int } p) (\text{int } q)$ in
 $\text{nat } ((\text{int } (x \wedge \text{calculate-exponent } (\text{length } c) \text{ } q \bmod q) * r-p * \text{int } p$
 $+ \text{int } (x \wedge \text{calculate-exponent } (\text{length } c) \text{ } p \bmod p) * r-q * \text{int } q) \bmod \text{int } (p$
 $* q))) =$
 $r \wedge 2 \bmod (p * q)$ **by algebra**

then have *all-collapsed*: $\text{nat } ((\text{int } (x \wedge \text{calculate-exponent } (\text{length } c) \text{ } q \bmod q) * \text{fst } (\text{fst } (\text{euclid-ext } (\text{int } p) (\text{int } q))) * \text{int } p$
 $+ \text{int } (x \wedge \text{calculate-exponent } (\text{length } c) \text{ } p \bmod p) * \text{snd } (\text{fst } (\text{euclid-ext } (\text{int } p) (\text{int } q))) * \text{int } q) \bmod \text{int } (p * q)) =$
 $r \wedge 2 \bmod (p * q)$
by (*metis* (*no-types*, *lifting*) *case-prod-conv prod.collapse*)

have *decrypt-alt* $p \text{ } q$ (*encrypt-alt* $(p * q) \text{ } m \text{ } r$) = *decrypt-alt* $p \text{ } q$ (c, x)
using *obtain-c-opt1 obtain-x-opt1 by auto*

also have ... =
(*case* (c, x) *of* $(c, x) \Rightarrow$
let $u-p = x \wedge \text{calculate-exponent } (\text{length } c) \text{ } p \bmod p$;
 $u-q = x \wedge \text{calculate-exponent } (\text{length } c) \text{ } q \bmod q$;
 $((r-p, r-q), -) = \text{euclid-ext } (\text{int } p) (\text{int } q)$;
 $x-0 = \text{nat } ((\text{int } u-q * r-p * \text{int } p + \text{int } u-p * r-q * \text{int } q) \bmod \text{int } (p * q))$;
 $(m, x') = \text{enc-loop-alt } (p * q, \text{nat } \lfloor \log 2 (\log 2 (p * q)) \rfloor, c) \text{ } x-0$
in *merge-list* m) **using** *decrypt-alt-def by force*

ultimately have *decrypt-alt* $p \text{ } q$ (*encrypt-alt* $(p * q) \text{ } m \text{ } r$) =
(let $u-p = x \wedge (\text{calculate-exponent } (\text{length } c) \text{ } p) \bmod p$;
 $u-q = x \wedge (\text{calculate-exponent } (\text{length } c) \text{ } q) \bmod q$;
 $((r-p, r-q), -) = \text{euclid-ext } p \text{ } q$;
 $x-0 = \text{nat } ((u-q * r-p * p + u-p * r-q * q) \bmod (p * q))$;
 $(m, x') = \text{enc-loop-alt } (p * q, (\text{nat } h), c) \text{ } x-0$ in
merge-list m) **using** *h-obtain by auto*

then have *decrypt-alt* $p \text{ } q$ (*encrypt-alt* $(p * q) \text{ } m \text{ } r$) =
(*let* $((r-p, r-q), -) = \text{euclid-ext } p \text{ } q$;
 $x-0 = \text{nat } (((x \wedge (\text{calculate-exponent } (\text{length } c) \text{ } q) \bmod q) * r-p * p$
 $+ (x \wedge (\text{calculate-exponent } (\text{length } c) \text{ } p) \bmod p) * r-q * q) \bmod (p * q))$;


```

    (m, x') = enc-loop-alt (p * q, (nat h) , c) x-0 in
      merge-list m)
  by algebra

then have decrypt-alt p q (encrypt-alt (p * q) m r) =
  (let
    x-0 = nat (((x ^ (calculate-exponent (length c) q) mod q) * fst (fst (euclid-ext
(int p) (int q)))) * p
    + (x ^ (calculate-exponent (length c) p) mod p) * snd (fst (euclid-ext (int
p) (int q)))) * q) mod (p * q));
    (m, x') = enc-loop-alt (p * q, (nat h) , c) x-0 in
      merge-list m)
  by (metis (no-types, lifting) case-prod-conv prod.collapse)

then have decrypt-alt p q (encrypt-alt (p * q) m r) = (let
  x-0 = r ^ 2 mod (p * q);
  (m, x') = enc-loop-alt (p * q, (nat h) , c) x-0 in
    merge-list m) using all-collapsed by algebra
then have decrypt-eq-merge: decrypt-alt p q (encrypt-alt (p * q) m r) =
  merge-list (fst (enc-loop-alt (p * q, (nat h) , c) (r ^ 2 mod (p * q)))) by auto

from assms(1,2,3,4) have pq-greater-zero: p * q > 0 by auto
then have p-greater-zero: p > 0 by auto
from pq-greater-zero have q-greater-zero: q > 0 by auto

have not-prime-1: ¬ prime 1 by auto
then have p-noteq-1: p ≠ 1 using assms(3) by auto
from not-prime-1 have q-noteq-1: q ≠ 1 using assms(4) by auto
from assms(1) have p-noteq-2: p ≠ 2 by linarith
from assms(2) have q-noteq-2: q ≠ 2 by linarith
from p-noteq-1 p-noteq-2 p-greater-zero have p-gr-2: p > 2 by auto
from q-noteq-1 q-noteq-2 q-greater-zero have q-gr-2: q > 2 by auto

from p-gr-2 q-gr-2 have p * q > 2 * 2
  using mult-strict-mono by blast
then have p * q > 2 ^ 2 by auto
then have log 2 (p * q) > 2
  using Multiseries-Expansion.intyness-simps(6) less-log2-of-power by presburger
then have log 2 (log 2 (p * q)) > 1 by auto
then have ⌊log 2 (log 2 (real (p * q)))⌋ ≥ 1 by auto
then have nat ⌊log 2 (log 2 (real (p * q)))⌋ ≥ 1
  by (simp add: le-simps(3))
then have h-gr-1: h ≥ 1 using h-obtain by auto

from this assms(7,8) have r ≠ 0
  using coprime-crossproduct-nat coprime-mult-right-iff mult-eq-if
  by (metis)
then have r > 0 by auto

```

```

from this assms have x-gr-0:  $r \wedge 2 \bmod (p * q) > 0$ 
by (auto simp add: coprime-commute coprime-dvd-mult-left-iff mod-greater-zero-iff-not-dvd
    nat-dvd-not-less power2-eq-square)

have x-less-n:  $r \wedge 2 \bmod (p * q) < p * q$ 
using mod-less-divisor pq-greater-zero by blast

from split-l have split-set:  $\forall l \in \text{set } (\text{split } m \ h). \text{length } l \leq h$  by auto

from x-gr-0 pq-greater-zero h-gr-1 x-less-n split-set enc-loop-alt-correctness
have  $c = \text{fst } (\text{enc-loop-alt } (p * q, h, (\text{split } m \ h)) (r \wedge 2 \bmod (p * q))) \implies$ 
     $\text{fst } (\text{enc-loop-alt } (p * q, (\text{nat } h), c) (r \wedge 2 \bmod (p * q))) = (\text{split } m \ (\text{nat } h))$ 
by auto

then have  $c = (\text{enc-loop-func-list } (\text{nat-to-bitstring } h, p * q, h, (\text{split } m \ h)) (r \wedge$ 
 $2 \bmod (p * q))) \implies$ 
     $\text{fst } (\text{enc-loop-alt } (p * q, (\text{nat } h), c) (r \wedge 2 \bmod (p * q))) = (\text{split } m \ (\text{nat } h))$ 
by auto

then have  $\text{fst } (\text{enc-loop-alt } (p * q, (\text{nat } h), c) (r \wedge 2 \bmod (p * q))) = (\text{split } m$ 
 $(\text{nat } h))$ 
using obtain-c-opt3 power2-eq-square[symmetric] by metis

then have decrypt-merge:  $\text{decrypt-alt } p \ q \ (\text{encrypt-alt } (p * q) \ m \ r) =$ 
     $\text{merge-list } ((\text{split } m \ (\text{nat } h)))$  using decrypt-eq-merge by auto

from h-gr-1 have  $\text{merge-list } ((\text{split } m \ (\text{nat } h))) = m$ 
using merge-split-in-place by auto

from this decrypt-merge show ?thesis by auto
qed

```

Showing the equivalence of *enc-loop* and *enc-loop-alt*, which splits the job into two different functions, *enc-loop-func-x* and *enc-loop-func-list*.

```

lemma enc-loop-func-exp:
  assumes  $n > 0 \ x < n$ 
  shows  $\text{snd } (\text{enc-loop-func } (\text{nat-to-bitstring } h, n, h, m) \ c \ x) = x \wedge (2 \wedge (\text{length } m)) \bmod n$ 
  using assms
proof (induction m arbitrary: x c)
  case Nil
  then have  $(\text{enc-loop-func } (\text{nat-to-bitstring } h, n, h, []) \ c \ x) = (\text{rev } c, x)$  using
enc-loop-func.simps(1) by auto
  then show ?case using local.Nil(2) by auto
next
  case (Cons a m)
  then have enc-loop-def:  $\text{snd } (\text{enc-loop-func } (\text{nat-to-bitstring } h, n, h, (a \# m))$ 
 $c \ x) = \text{snd}(\text{enc-loop-func } (\text{nat-to-bitstring } h, n, h, m) \ c \ x)$ 

```

(let $x-i = x * x \bmod n$ in
 let $p-i = (\text{nat-to-bitstring } h) (x-i \bmod 2^h)$ in
 let $c-i = a [\oplus] p-i$ in
 $\text{enc-loop-func } ((\text{nat-to-bitstring } h), n, h, m) (c-i \# c) x-i))$ **using** $\text{enc-loop-func.simps}$
by *auto*

from $\text{mod-less-divisor } \text{assms}(1)$ **have** $x * x \bmod n < n$ **by** *auto*

from $\text{this } \text{enc-loop-def } \text{Cons.IH } \text{assms}(1) \text{ assms}(2)$
have $\text{use-ia: snd } (\text{enc-loop-func } (\text{nat-to-bitstring } h, n, h, (a \# m)) c x) = (x * x \bmod n) \wedge (2 \wedge \text{length } m) \bmod n$
by *metis*

have $(x * x) \wedge (2 \wedge \text{length } m) \bmod n = (x * x \bmod n) \wedge (2 \wedge \text{length } m) \bmod n$
using $\text{power-mod[symmetric]}$ **by** *auto*
from $\text{use-ia this have } x\text{-times-}x$: $\text{snd } (\text{enc-loop-func } (\text{nat-to-bitstring } h, n, h, (a \# m)) c x) = (x * x) \wedge (2 \wedge \text{length } m) \bmod n$
by *auto*

have $(x * x) \wedge (2 \wedge \text{length } m) = x \wedge (2 \wedge \text{length } (a \# m))$
by $(\text{auto simp add: power2-eq-square power-mult})$
from $\text{this } x\text{-times-}x$ **show** $?case$ **by** *auto*
qed

corollary enc-loop-x :

assumes $n > 0 \ x < n$
shows $\text{snd } (\text{enc-loop-func } (\text{nat-to-bitstring } h, n, h, m) c x) = \text{snd } (\text{enc-loop-alt } (n, h, m) x)$
by $(\text{auto intro: enc-loop-func-exp simp add: assms})$

corollary $\text{enc-loop-x-enc-loop-normal}$:

assumes $n > 0 \ x < n$
shows $\text{snd } (\text{enc-loop } (n, h, m) x) = \text{snd } (\text{enc-loop-alt } (n, h, m) x)$
by $(\text{auto intro: enc-loop-func-exp simp add: assms})$

corollary enc-loop-exp :

assumes $n > 0 \ h > 0 \ x < n$
shows $\text{snd } (\text{enc-loop } (n, h, m) x) = x \wedge (2 \wedge (\text{length } m)) \bmod n$
by $(\text{auto intro: enc-loop-func-exp simp add: assms})$

theorem enc-x-exp :

assumes $n > 0 \ r < n \ h = \text{nat } \lfloor \log 2 (\log 2 n) \rfloor \ m-l = \text{split } m \ h$
shows $\text{snd } (\text{encrypt } n \ m \ r) = (r * r) \wedge (2 \wedge (\text{length } m-l + 1)) \bmod n$
proof –
from encrypt-def **have** $\text{encrypt-open: snd } (\text{encrypt } n \ m \ r) = \text{snd } ($
 let $h = \text{nat } \lfloor \log 2 (\log 2 n) \rfloor$ in
 let $m-l = \text{split } m \ h$ in
 let $(x-0 :: \text{nat}) = (r * r) \bmod n$ in
 let $(c, x-t) = \text{enc-loop } (n, h, m-l) x-0$ in

```

let x-t' = x-t * x-t mod n in
(c, x-t')) by auto

have r * r mod n < n using mod-less-divisor assms(1) by auto

then have snd (
  let h = nat ⌊log 2 (log 2 n)⌋ in
  let m-l = split m h in
  let (x-0 :: nat) = (r * r) mod n in
  enc-loop (n, h, m-l) x-0 =
  (let h = nat ⌊log 2 (log 2 n)⌋ in
   let m-l = split m h in
   let (x-0 :: nat) = (r * r) mod n in
   (x-0 ^ (2 ^ (length m-l)) mod n)) using assms(1) enc-loop.simps enc-loop-func-exp
  by (metis)
from encrypt-open this have snd (
  let h = nat ⌊log 2 (log 2 n)⌋ in
  let m-l = split m h in
  let (x-0 :: nat) = (r * r) mod n in
  let (c, x-t) = enc-loop (n, h, m-l) x-0 in
  let x-t' = x-t * x-t mod n in
  (c, x-t'))
=
  (let h = nat ⌊log 2 (log 2 n)⌋ in
   let m-l = split m h in
   let (x-0 :: nat) = (r * r) mod n in
   let x-t' = x-0 ^ (2 ^ (length m-l)) mod n in
   x-t' * x-t' mod n)
using old.prod.exhaust snd-conv split-conv
by (smt (verit, best))

then have snd (
  let h = nat ⌊log 2 (log 2 n)⌋ in
  let m-l = split m h in
  let (x-0 :: nat) = (r * r) mod n in
  let (c, x-t) = enc-loop (n, h, m-l) x-0 in
  let x-t' = x-t * x-t mod n in
  (c, x-t'))
=
  (((r * r) mod n) ^ (2 ^ (length m-l)) mod n) * (((r * r) mod n) ^ (2 ^ (length
m-l)) mod n) mod n
using assms
by meson

then have snd (encrypt n m r) = (((r * r) mod n) ^ (2 ^ (length m-l)) mod n)
* (((r * r) mod n) ^ (2 ^ (length m-l)) mod n) mod n
using encrypt-open by auto
then have snd (encrypt n m r) = ((r * r) ^ (2 ^ (length m-l)) mod n) * ((r *
r) ^ (2 ^ (length m-l)) mod n) mod n

```

```

using power-mod by metis
then have encrypt-to-exp: snd (encrypt n m r) = (((r * r) ^ (2 ^ (length m-l)))
* ((r * r) ^ (2 ^ (length m-l)))) mod n
by (simp add: mod-mult-eq)

have (((r * r) ^ (2 ^ (length m-l))) * ((r * r) ^ (2 ^ (length m-l)))) = (r * r)
^ (2 ^ (length m-l) + 2 ^ (length m-l))
using power-add[symmetric] by auto
then have (((r * r) ^ (2 ^ (length m-l))) * ((r * r) ^ (2 ^ (length m-l)))) = (r
* r) ^ (2 * 2 ^ (length m-l))
by (simp add: mult-2)
then have (((r * r) ^ (2 ^ (length m-l))) * ((r * r) ^ (2 ^ (length m-l)))) = (r
* r) ^ (2 ^ (length m-l + 1))
by auto
from this encrypt-to-exp show ?thesis by auto
qed

```

```

lemma fst-enc-loop-eq-enc-between: fst (enc-loop-func (f, n, h, m) c-acc x) =
enc-loop-func-between (f, n, h, m) c-acc x
proof (induction m arbitrary: x c-acc)
case (Nil)
then show ?case by auto
next
case (Cons a m)
have fst (enc-loop-func (f, n, h, (a # m)) c-acc x) = fst (let x-i = x * x mod
n in
  let p-i = f (x-i mod 2^h) in
  let c-i = a [⊕] p-i in
  enc-loop-func (f, n, h, m) (c-i # c-acc) x-i
)
by auto
then have fst (enc-loop-func (f, n, h, (a # m)) c-acc x) = (let x-i = x * x mod
n in
  let p-i = f (x-i mod 2^h) in
  let c-i = a [⊕] p-i in
  fst (enc-loop-func (f, n, h, m) (c-i # c-acc) x-i)
) unfolding Let-def by auto
then have fst (enc-loop-func (f, n, h, (a # m)) c-acc x) = (let x-i = x * x
mod n in
  let p-i = f (x-i mod 2^h) in
  let c-i = a [⊕] p-i in
  enc-loop-func-between (f, n, h, m) (c-i # c-acc) x-i
) using Cons by auto
then show ?case by auto
qed

```

```

lemma enc-betw-eq-enc-list: enc-loop-func-between (f, n, h, m) c-acc x = rev c-acc
@ (enc-loop-func-list (f, n, h, m) x)
proof (induction m arbitrary: x c-acc)

```

```

case Nil
then show ?case by auto
next
  case (Cons a m)
  let ?c-i = a [⊕] f ((x * x mod n) mod 2^h)
  let ?x-i = x * x mod n
  have enc-loop-func-between (f, n, h, a # m) c-acc x = (let x-i = x * x mod n
in
  let p-i = f (x-i mod 2^h) in
  let c-i = a [⊕] p-i in
  enc-loop-func-between (f, n, h, m) (c-i # c-acc) x-i
) by auto
  hence enc-loop-func-between (f, n, h, a # m) c-acc x = enc-loop-func-between
(f, n, h, m) (?c-i # c-acc) ?x-i
  unfolding Let-def by auto
  hence enc-loop-func-between (f, n, h, a # m) c-acc x = rev (?c-i # c-acc) @
enc-loop-func-list (f, n, h, m) ?x-i
  using Cons by auto
  hence enc-loop-func-between (f, n, h, a # m) c-acc x = rev c-acc @ [?c-i] @
enc-loop-func-list (f, n, h, m) ?x-i
  using rev-def by auto
  hence enc-loop-betw-def: enc-loop-func-between (f, n, h, a # m) c-acc x = rev
c-acc @ (?c-i # enc-loop-func-list (f, n, h, m) ?x-i)
  by auto
  have enc-loop-func-list (f, n, h, a # m) x = (let x-i = x * x mod n in
  let p-i = f (x-i mod 2^h) in
  let c-i = a [⊕] p-i in
  c-i # enc-loop-func-list (f, n, h, m) x-i
) by auto
  hence enc-loop-func-list (f, n, h, a # m) x = ?c-i # enc-loop-func-list (f, n,
h, m) (x * x mod n)
  unfolding Let-def by auto
  thus ?case using enc-loop-betw-def unfolding Let-def by auto
qed

```

corollary fst-enc-eq-enc-list: $\text{fst} (\text{enc-loop-func} (f, n, h, m) [] x) = \text{enc-loop-func-list} (f, n, h, m) x$
using enc-betw-eq-enc-list fst-enc-loop-eq-enc-between **by auto**

corollary

```

assumes 0 < n x < n
shows enc-loop (n, h, m) x = enc-loop-alt (n, h, m) x
using enc-loop-x-enc-loop-normal fst-enc-eq-enc-list
proof -
  have snd (enc-loop (n, h, m) x) = snd (enc-loop-func (nat-to-bitstring h, n, h,
m) [] x)
  by auto
  then have snd-eq: snd (enc-loop (n, h, m) x) = snd (enc-loop-alt (n, h, m) x)
  using assms enc-loop-x-enc-loop-normal by auto

```

```

    have fst (enc-loop (n, h, m) x) = fst (enc-loop-func (nat-to-bitstring h, n, h, m)
    [] x)
      by auto
    then have fst (enc-loop (n, h, m) x) = enc-loop-func-list (nat-to-bitstring h, n,
    h, m) x
      using fst-enc-eq-enc-list by auto
    then have fst-eq: fst (enc-loop (n, h, m) x) = fst (enc-loop-alt (n, h, m) x) by
    auto

    from fst-eq snd-eq show ?thesis
      using prod.expand by blast
qed
end
theory BG-prob imports
  Main
  BG-aux
  BG-formalization
  BG-correctness
begin

```

6 Formalization of Blum-Goldwasser cryptosystem in a probabilistic setting

```

type-synonym pub-key = nat
type-synonym priv-key = nat × nat

```

6.1 Key generation

We generate a key by uniformly sampling a set of natural numbers that are congruent to 3 modulo 4. To ensure that $p \neq q$, we remove the first selected number from the sample set.

definition *key-gen-prob* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow (\text{pub-key} \times \text{priv-key}) \text{ pmf}$
where

```

  key-gen-prob lower upper = do {
    let  $\mathcal{K} = \{(p::\text{nat}). \text{prime } p \wedge p \bmod 4 = 3 \wedge \text{lower} \leq p \wedge p \leq \text{upper}\}$ ;
     $p \leftarrow \text{pmf-of-set } \mathcal{K}$ ;
     $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$ ;
    return-pmf (key-gen p q)
  }

```

definition *encrypt-prob* :: $\text{pub-key} \Rightarrow \text{bitstring} \Rightarrow (\text{bitstring list} \times \text{nat}) \text{ pmf}$ **where**
encrypt-prob n m = do {
 let $\mathcal{Z} = \{(z::\text{nat}). z < n \wedge \text{coprime } z \ n\}$;
 $r \leftarrow \text{pmf-of-set } \mathcal{Z}$;

```

    return-pmf (encrypt-alt n m r)
  }

```

definition *decrypt-prob* :: *priv-key* $\Rightarrow$ (*bitstring list* $\times$ *nat*) $\Rightarrow$ *bitstring pmf* **where**
decrypt-prob = ($\lambda(p,q)$ *c*. return-pmf (decrypt-alt p q c))

theorem

assumes

$m \neq []$

$\text{card } \{(p::\text{nat}). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\} > 1$

shows pmf (do {

($n, (p, q)$) $\leftarrow$ key-gen-prob l u;

(c, x) $\leftarrow$ (encrypt-prob n m);

return-pmf (decrypt-alt p q (c, x) = m)) True = 1

proof –

have do {

($n, (p, q)$) $\leftarrow$ key-gen-prob l u;

(c, x) $\leftarrow$ (encrypt-prob n m);

return-pmf (decrypt-alt p q (c, x) = m)} =

do {

($n, (p, q)$) $\leftarrow$ key-gen-prob l u;

let $\mathcal{Z} = \{(z::\text{nat}). z < n \wedge \text{coprime } z \ n\}$;

$r \leftarrow$ pmf-of-set $\mathcal{Z}$;

(c, x) $\leftarrow$ return-pmf (encrypt-alt n m r);

return-pmf (decrypt-alt p q (c, x) = m)}

unfolding encrypt-prob-def

by (auto simp add: bind-assoc-pmf)

also have ... =

do {

($n, (p, q)$) $\leftarrow$ key-gen-prob l u;

let $\mathcal{Z} = \{(z::\text{nat}). z < n \wedge \text{coprime } z \ n\}$;

$r \leftarrow$ pmf-of-set $\mathcal{Z}$;

return-pmf (decrypt-alt p q (encrypt-alt n m r) = m)}

by (auto simp add: bind-return-pmf)

also have ... =

do {

($n, (p, q)$) $\leftarrow$ do {

let $\mathcal{K} = \{(p::\text{nat}). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$;

$p \leftarrow$ pmf-of-set $\mathcal{K}$;

$q \leftarrow$ pmf-of-set ($\mathcal{K} - \{p\}$);

return-pmf (key-gen p q)

};

let $\mathcal{Z} = \{(z::\text{nat}). z < n \wedge \text{coprime } z \ n\}$;

$r \leftarrow$ pmf-of-set $\mathcal{Z}$;

return-pmf (decrypt-alt p q (encrypt-alt n m r) = m)}

unfolding *key-gen-prob-def*
by *auto*

also have ... =
 do {
 let $\mathcal{K} = \{(p::nat). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$;
 $p \leftarrow \text{pmf-of-set } \mathcal{K}$;
 $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$;
 $(n, (p, q)) \leftarrow \text{return-pmf } (\text{key-gen } p \ q)$;
 let $\mathcal{Z} = \{(z::nat). z < n \wedge \text{coprime } z \ n\}$;
 $r \leftarrow \text{pmf-of-set } \mathcal{Z}$;
 return-pmf (decrypt-alt $p \ q$ (encrypt-alt $n \ m \ r$) = m)}
unfolding *Let-def*
by (*auto simp add: bind-assoc-pmf*)

also have ... =
 do {
 let $\mathcal{K} = \{(p::nat). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$;
 $p \leftarrow \text{pmf-of-set } \mathcal{K}$;
 $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$;
 $(n, (p, q)) \leftarrow \text{return-pmf } ((p * q, (p, q)))$;
 let $\mathcal{Z} = \{(z::nat). z < n \wedge \text{coprime } z \ n\}$;
 $r \leftarrow \text{pmf-of-set } \mathcal{Z}$;
 return-pmf (decrypt-alt $p \ q$ (encrypt-alt $n \ m \ r$) = m)}
unfolding *key-gen-def*
by *auto*

also have ... =
 do {
 let $\mathcal{K} = \{(p::nat). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$;
 $p \leftarrow \text{pmf-of-set } \mathcal{K}$;
 $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$;
 let $\mathcal{Z} = \{(z::nat). z < p * q \wedge \text{coprime } z \ (p * q)\}$;
 $r \leftarrow \text{pmf-of-set } \mathcal{Z}$;
 return-pmf (decrypt-alt $p \ q$ (encrypt-alt $(p * q) \ m \ r$) = m)}
by (*auto simp add: bind-return-pmf*)

from *calculation* **have** do {
 $(n, (p, q)) \leftarrow \text{key-gen-prob } l \ u$;
 $(c, x) \leftarrow (\text{encrypt-prob } n \ m)$;
 return-pmf (decrypt-alt $p \ q$ (c, x) = m)}
 = do {
 let $\mathcal{K} = \{(p::nat). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$;
 $p \leftarrow \text{pmf-of-set } \mathcal{K}$;
 $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$;
 let $\mathcal{Z} = \{(z::nat). z < p * q \wedge \text{coprime } z \ (p * q)\}$;
 $r \leftarrow \text{pmf-of-set } \mathcal{Z}$;
 return-pmf (decrypt-alt $p \ q$ (encrypt-alt $(p * q) \ m \ r$) = m)}
using *key-gen-def*

by (*auto simp add: bind-return-pmf*)

then have *step-1*:

```

pmf (do {
  (n, (p, q)) ← key-gen-prob l u;
  (c, x) ← (encrypt-prob n m);
  return-pmf (decrypt-alt p q (c, x) = m)})
=
pmf (do {
  let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
  p ← pmf-of-set K;
  q ← pmf-of-set (K - {p});
  let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
  r ← pmf-of-set Z;
  return-pmf (decrypt-alt p q (encrypt-alt (p * q) m r) = m)})
by auto

```

then have

```

pmf (do {
  let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
  p ← pmf-of-set K;
  q ← pmf-of-set (K - {p});
  let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
  r ← pmf-of-set Z;
  return-pmf (decrypt-alt p q (encrypt-alt (p * q) m r) = m)})
=
pmf (do {
  let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
  p ← pmf-of-set K;
  q ← pmf-of-set (K - {p});
  let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
  r ← pmf-of-set Z;
  return-pmf ((prime p → decrypt-alt p q (encrypt-alt (p * q) m r) = m) ∧
  (¬prime p → decrypt-alt p q (encrypt-alt (p * q) m r) = m)))
by auto

```

have *correctness-int*: $(\text{prime } p \wedge \text{prime } q \wedge p \bmod 4 = 3 \wedge q \bmod 4 = 3 \wedge r < p * q \wedge \text{coprime } r (p * q) \wedge p \neq q \wedge m \neq [] \longrightarrow \text{decrypt-alt } p \ q \ (\text{encrypt-alt } (p * q) \ m \ r) = m)$

using *correctness*

by *auto*

have $A = ((B \longrightarrow A) \wedge (\neg B \longrightarrow A))$

by *auto*

then have *iff-rule*: $(\text{decrypt-alt } p \ q \ (\text{encrypt-alt } (p * q) \ m \ r) = m) =$

$((\text{prime } p \wedge \text{prime } q \wedge p \bmod 4 = 3 \wedge q \bmod 4 = 3 \wedge r < p * q \wedge \text{coprime } r (p * q) \wedge p \neq q \wedge m \neq []) \longrightarrow \text{decrypt-alt } p \ q \ (\text{encrypt-alt } (p * q) \ m \ r) = m)$

$\wedge (\neg(\text{prime } p \wedge \text{prime } q \wedge p \bmod 4 = 3 \wedge q \bmod 4 = 3 \wedge r < p * q \wedge \text{coprime } r (p * q) \wedge p \neq q \wedge m \neq []) \longrightarrow \text{decrypt-alt } p \ q (\text{encrypt-alt } (p * q) \ m \ r) = m))$

by *auto*

let $?K = \{(p::\text{nat}). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$
let $?Z = \{(z::\text{nat}). z < p * q \wedge \text{coprime } z (p * q)\}$

have $K\text{-finite}: \text{finite } ?K$

by *auto*

have $K\text{-not-empty}: ?K \neq \{\}$

using $\text{assms}(2)$

by *fastforce*

from $K\text{-finite } K\text{-not-empty}$ **have** $K\text{-finite}' : \text{finite}' ?K$

by *auto*

have $K\text{-not-empty-after-}p: \bigwedge p. ?K - \{p\} \neq \{\}$

using $\text{assms}(2)$

by (*metis* (*no-types*, *lifting*) $K\text{-finite}' \text{ card-Diff-singleton-if card-gt-0-iff zero-less-diff}$)

then have $K\text{-card-after-}p: \text{card } ?K - 1 > 0$

using $\text{assms}(2)$ **by** *linarith*

from this $K\text{-finite}$ **have** $K\text{-finite-after-}p: \bigwedge p. \text{finite } (?K - \{p\})$

by *auto*

then have

$\text{pmf } (do \{$

$\text{let } K = \{(p::\text{nat}). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\};$

$p \leftarrow \text{pmf-of-set } K;$

$q \leftarrow \text{pmf-of-set } (K - \{p\});$

$\text{let } Z = \{(z::\text{nat}). z < p * q \wedge \text{coprime } z (p * q)\};$

$r \leftarrow \text{pmf-of-set } Z;$

$\text{return-pmf } (\text{decrypt-alt } p \ q (\text{encrypt-alt } (p * q) \ m \ r) = m)) \text{ True}$

$=$

$\text{pmf } (do \{$

$\text{let } K = \{(p::\text{nat}). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\};$

$p \leftarrow \text{pmf-of-set } K;$

$q \leftarrow \text{pmf-of-set } (K - \{p\});$

$\text{let } Z = \{(z::\text{nat}). z < p * q \wedge \text{coprime } z (p * q)\};$

$r \leftarrow \text{pmf-of-set } Z;$

$\text{return-pmf } (((\text{prime } p \wedge \text{prime } q \wedge p \bmod 4 = 3 \wedge q \bmod 4 = 3 \wedge r < p * q \wedge \text{coprime } r (p * q) \wedge p \neq q \wedge m \neq []) \longrightarrow \text{decrypt-alt } p \ q (\text{encrypt-alt } (p * q) \ m \ r) = m))$

$\wedge (\neg(\text{prime } p \wedge \text{prime } q \wedge p \bmod 4 = 3 \wedge q \bmod 4 = 3 \wedge r < p * q \wedge \text{coprime } r (p * q) \wedge p \neq q \wedge m \neq []) \longrightarrow \text{decrypt-alt } p \ q (\text{encrypt-alt } (p * q) \ m \ r) = m))))$

True

using $\text{iff-rule}[\text{symmetric}]$

```

    by metis
  then have
    pmf (do {
      let  $\mathcal{K} = \{(p::nat). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$ ;
       $p \leftarrow \text{pmf-of-set } \mathcal{K}$ ;
       $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$ ;
      let  $\mathcal{Z} = \{(z::nat). z < p * q \wedge \text{coprime } z (p * q)\}$ ;
       $r \leftarrow \text{pmf-of-set } \mathcal{Z}$ ;
      return-pmf (decrypt-alt  $p \ q$  (encrypt-alt ( $p * q$ )  $m \ r$ ) =  $m$ ))} True
    =
    pmf (do {
      let  $\mathcal{K} = \{(p::nat). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$ ;
       $p \leftarrow \text{pmf-of-set } \mathcal{K}$ ;
       $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$ ;
      let  $\mathcal{Z} = \{(z::nat). z < p * q \wedge \text{coprime } z (p * q)\}$ ;
       $r \leftarrow \text{pmf-of-set } \mathcal{Z}$ ;
      return-pmf (( $\neg(\text{prime } p \wedge \text{prime } q \wedge p \bmod 4 = 3 \wedge q \bmod 4 = 3 \wedge r < p * q$ 
 $\wedge \text{coprime } r (p * q) \wedge p \neq q \wedge m \neq [])$   $\longrightarrow$  decrypt-alt  $p \ q$  (encrypt-alt ( $p * q$ )  $m \ r$ )
      =  $m$ )))}
    True
    using correctness
    by algebra

  then have
    pmf (do {
      let  $\mathcal{K} = \{(p::nat). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$ ;
       $p \leftarrow \text{pmf-of-set } \mathcal{K}$ ;
       $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$ ;
      let  $\mathcal{Z} = \{(z::nat). z < p * q \wedge \text{coprime } z (p * q)\}$ ;
       $r \leftarrow \text{pmf-of-set } \mathcal{Z}$ ;
      return-pmf (decrypt-alt  $p \ q$  (encrypt-alt ( $p * q$ )  $m \ r$ ) =  $m$ ))} True
    =
    pmf (do {
      let  $\mathcal{K} = \{(p::nat). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$ ;
       $p \leftarrow \text{pmf-of-set } \mathcal{K}$ ;
       $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$ ;
      let  $\mathcal{Z} = \{(z::nat). z < p * q \wedge \text{coprime } z (p * q)\}$ ;
       $r \leftarrow \text{pmf-of-set } \mathcal{Z}$ ;
      return-pmf (( $\text{prime } p \wedge \text{prime } q \wedge p \bmod 4 = 3 \wedge q \bmod 4 = 3 \wedge r < p * q \wedge$ 
 $\text{coprime } r (p * q) \wedge p \neq q \wedge m \neq []$ )  $\vee$  decrypt-alt  $p \ q$  (encrypt-alt ( $p * q$ )  $m \ r$ )
      =  $m$ )))}
    True
    by linarith
  then have
    pmf (do {
      let  $\mathcal{K} = \{(p::nat). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\}$ ;
       $p \leftarrow \text{pmf-of-set } \mathcal{K}$ ;
       $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\})$ ;
      let  $\mathcal{Z} = \{(z::nat). z < p * q \wedge \text{coprime } z (p * q)\}$ ;

```

```

r ← pmf-of-set Z;
return-pmf (decrypt-alt p q (encrypt-alt (p * q) m r) = m)) True
=
pmf (do {
let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
p ← pmf-of-set K;
q ← pmf-of-set (K - {p});
let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
r ← pmf-of-set Z;
return-pmf ((prime p ∧ prime q ∧ p mod 4 = 3 ∧ q mod 4 = 3 ∧ r < p * q ∧
coprime r (p * q) ∧ p ≠ q) ∨ decrypt-alt p q (encrypt-alt (p * q) m r) = m))
True
using assms(1) by auto

```

then have

```

pmf (do {
let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
p ← pmf-of-set K;
q ← pmf-of-set (K - {p});
let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
r ← pmf-of-set Z;
return-pmf (decrypt-alt p q (encrypt-alt (p * q) m r) = m)) True
=
pmf (do {
let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
p ← pmf-of-set K;
q ← pmf-of-set (K - {p});
let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
r ← pmf-of-set Z;
return-pmf (((p ∈ K ∧ q ∈ K - {p} ∧ r ∈ Z) → prime p ∧ prime q ∧ p mod
4 = 3 ∧ q mod 4 = 3 ∧ r < p * q ∧ coprime r (p * q) ∧ p ≠ q)
∧ (¬(p ∈ K ∧ q ∈ K - {p} ∧ r ∈ Z) → prime p ∧ prime q ∧ p mod 4 = 3 ∧
q mod 4 = 3 ∧ r < p * q ∧ coprime r (p * q) ∧ p ≠ q))
∨ decrypt-alt p q (encrypt-alt (p * q) m r) = m))
True
by metis

```

then have

```

pmf (do {
let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
p ← pmf-of-set K;
q ← pmf-of-set (K - {p});
let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
r ← pmf-of-set Z;
return-pmf (decrypt-alt p q (encrypt-alt (p * q) m r) = m)) True
=
pmf (do {
let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
p ← pmf-of-set K;

```

```

  q ← pmf-of-set (K - {p});
  let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
  r ← pmf-of-set Z;
  return-pmf (((¬(p ∈ K ∧ q ∈ K - {p}) ∧ r ∈ Z) → prime p ∧ prime q ∧ p
mod 4 = 3 ∧ q mod 4 = 3 ∧ r < p * q ∧ coprime r (p * q) ∧ p ≠ q))
  ∨ decrypt-alt p q (encrypt-alt (p * q) m r) = m))
  True
  by simp

```

then have *spmf-last*:

```

pmf (do {
  let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
  p ← pmf-of-set K;
  q ← pmf-of-set (K - {p});
  let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
  r ← pmf-of-set Z;
  return-pmf (decrypt-alt p q (encrypt-alt (p * q) m r) = m)) True
=
pmf (do {
  let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
  p ← pmf-of-set K;
  q ← pmf-of-set (K - {p});
  let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
  r ← pmf-of-set Z;
  return-pmf (((p ∈ K ∧ q ∈ K - {p}) ∧ r ∈ Z) ∨ (prime p ∧ prime q ∧ p mod 4
= 3 ∧ q mod 4 = 3 ∧ r < p * q ∧ coprime r (p * q) ∧ p ≠ q))
  ∨ decrypt-alt p q (encrypt-alt (p * q) m r) = m))
  True
  by linarith

```

```

let ?pmf = (do {
  p ← pmf-of-set ?K;
  q ← pmf-of-set (?K - {p});
  let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
  r ← pmf-of-set Z;
  return-pmf (((p ∈ ?K ∧ q ∈ ?K - {p}) ∧ r ∈ Z) ∨ (prime p ∧ prime q ∧ p mod
4 = 3 ∧ q mod 4 = 3 ∧ r < p * q ∧ coprime r (p * q) ∧ p ≠ q))
  ∨ decrypt-alt p q (encrypt-alt (p * q) m r) = m))

```

```

have replace-pmf: pmf (do {
  let K = {(p::nat). prime p ∧ p mod 4 = 3 ∧ l ≤ p ∧ p ≤ u};
  p ← pmf-of-set K;
  q ← pmf-of-set (K - {p});
  let Z = {(z::nat). z < p * q ∧ coprime z (p * q)};
  r ← pmf-of-set Z;
  return-pmf (((p ∈ K ∧ q ∈ K - {p}) ∧ r ∈ Z) ∨ (prime p ∧ prime q ∧ p mod 4
= 3 ∧ q mod 4 = 3 ∧ r < p * q ∧ coprime r (p * q) ∧ p ≠ q))
  ∨ decrypt-alt p q (encrypt-alt (p * q) m r) = m))
  True = pmf ?pmf True

```

by *metis*

let $?pmf\text{-}fun = (\lambda p. pmf\text{-}of\text{-}set\ (\mathcal{K} - \{p\}) \gg=$
 $(\lambda q. let\ \mathcal{Z} = \{z. z < p * q \wedge coprime\ z\ (p * q)\}$
 $in\ pmf\text{-}of\text{-}set\ \mathcal{Z} \gg=$
 $(\lambda r. return\text{-}pmf$
 $((p \in \mathcal{K} \wedge q \in \mathcal{K} - \{p\} \wedge r \in \mathcal{Z} \vee$
 $prime\ p \wedge$
 $prime\ q \wedge$
 $p \bmod 4 = 3 \wedge q \bmod 4 = 3 \wedge r < p * q \wedge coprime\ r$
 $(p * q) \wedge p \neq q) \vee$
 $decrypt\text{-}alt\ p\ q\ (encrypt\text{-}alt\ (p * q)\ m\ r) = m)))$

let $?pmf\text{-}fun' = (\lambda p. pmf\text{-}of\text{-}set\ (\mathcal{K} - \{p\}) \gg=$
 $(\lambda q. let\ \mathcal{Z} = \{z. z < p * q \wedge coprime\ z\ (p * q)\}$
 $in\ pmf\text{-}of\text{-}set\ \mathcal{Z} \gg=$
 $(\lambda r. return\text{-}pmf$
 $((q \in \mathcal{K} - \{p\} \wedge r \in \mathcal{Z} \vee$
 $prime\ q \wedge$
 $q \bmod 4 = 3 \wedge r < p * q \wedge coprime\ r\ (p * q) \wedge p \neq q) \vee$
 $decrypt\text{-}alt\ p\ q\ (encrypt\text{-}alt\ (p * q)\ m\ r) = m)))$

have $step\text{-}3: pmf\ ?pmf\ True = (\sum_{xa \in \mathcal{K}. pmf\ (?pmf\text{-}fun\ xa)\ True) / real\ (card\ \mathcal{K})$
using $pmf\text{-}bind\text{-}pmf\text{-}of\text{-}set\ \mathcal{K}\text{-}finite\ \mathcal{K}\text{-}not\text{-}empty$
by $(simp\ add: pmf\text{-}bind\text{-}pmf\text{-}of\text{-}set)$
then have $pmf\ ?pmf\ True = (\sum_{xa \in \mathcal{K}. pmf\ (?pmf\text{-}fun'\ xa)\ True) / real\ (card\ \mathcal{K})$
by $auto$

let $?pmf\text{-}fun'' = \lambda xa. (\lambda q. let\ \mathcal{Z} = \{z. z < xa * q \wedge coprime\ z\ (xa * q)\}$
 $in\ pmf\text{-}of\text{-}set\ \mathcal{Z} \gg=$
 $(\lambda r. return\text{-}pmf$
 $((q \in \mathcal{K} - \{xa\} \wedge r \in \mathcal{Z} \vee$
 $prime\ q \wedge$
 $q \bmod 4 = 3 \wedge r < xa * q \wedge coprime\ r\ (xa * q) \wedge xa$
 $\neq q) \vee$
 $decrypt\text{-}alt\ xa\ q\ (encrypt\text{-}alt\ (xa * q)\ m\ r) = m)))$

have $\bigwedge xa. (pmf\ (?pmf\text{-}fun'\ xa)\ True) =$
 $(\sum_{xb \in (\mathcal{K} - \{xa\}). pmf\ ((?pmf\text{-}fun''\ xa)\ xb)\ True) / real\ (card\ (\mathcal{K} - \{xa\}))$
using $\mathcal{K}\text{-}finite\text{-}after\text{-}p\ \mathcal{K}\text{-}not\text{-}empty\text{-}after\text{-}p$
by $(auto\ simp\ add: pmf\text{-}bind\text{-}pmf\text{-}of\text{-}set)$

then have $(\sum_{xa \in \mathcal{K}. pmf\ (?pmf\text{-}fun'\ xa)\ True) =$
 $(\sum_{xa \in \mathcal{K}. (\sum_{xb \in (\mathcal{K} - \{xa\}). pmf\ ((?pmf\text{-}fun''\ xa)\ xb)\ True) / real\ (card\ (\mathcal{K} - \{xa\})))$
by $auto$

then have $(\sum xa \in ?\mathcal{K}. \text{pmf } (?pmf\text{-fun}' xa) \text{ True}) =$
 $(\sum xa \in ?\mathcal{K}. (\sum xb \in (?\mathcal{K} - \{xa\}). \text{pmf } ((?pmf\text{-fun}'' xa) xb) \text{ True})) / \text{real } (\text{card } ?\mathcal{K} - 1)$
by *(simp add: sum-divide-distrib)*

then have $q\text{-escaped}: (\sum xa \in ?\mathcal{K}. \text{pmf } (?pmf\text{-fun}' xa) \text{ True}) / \text{real } (\text{card } ?\mathcal{K}) =$
 $(\sum xa \in ?\mathcal{K}. (\sum xb \in (?\mathcal{K} - \{xa\}). \text{pmf } ((?pmf\text{-fun}'' xa) xb) \text{ True})) / (\text{real } (\text{card } ?\mathcal{K} - 1) * \text{real } (\text{card } ?\mathcal{K}))$
by *auto*

let $?pmf\text{-fun}''' = \lambda xa. \lambda q. (\text{let } \mathcal{Z} = \{z. z < xa * q \wedge \text{coprime } z (xa * q)\}$
 $\text{in } pmf\text{-of-set } \mathcal{Z} \gg=$
 $(\lambda r. \text{return-pmf}$
 $((r \in \mathcal{Z} \vee$
 $r < xa * q \wedge \text{coprime } r (xa * q) \wedge xa \neq q) \vee$
 $\text{decrypt-alt } xa \ q \ (\text{encrypt-alt } (xa * q) \ m \ r) = m)))$

let $?Z = \lambda xa \ xb. \{(z::nat). z < xa * xb \wedge \text{coprime } z (xa * xb)\}$

let $?pmf\text{-fun-3alt} = \lambda xa. \lambda q. (pmf\text{-of-set } (?Z \ xa \ q) \gg=$
 $(\lambda r. \text{return-pmf}$
 $((r \in (?Z \ xa \ q) \vee$
 $r < xa * q \wedge \text{coprime } r (xa * q) \wedge xa \neq q) \vee$
 $\text{decrypt-alt } xa \ q \ (\text{encrypt-alt } (xa * q) \ m \ r) = m)))$

let $?pmf\text{-fun-4} = \lambda xa \ xb.$
 $(\lambda r. (\text{return-pmf}$
 $((r \in (?Z \ xa \ xb) \vee$
 $r < xa * xb \wedge \text{coprime } r (xa * xb) \wedge xa \neq xb) \vee$
 $\text{decrypt-alt } xa \ xb \ (\text{encrypt-alt } (xa * xb) \ m \ r) = m)))$

from $q\text{-escaped}$ **have** $ff: (\sum xa \in ?\mathcal{K}. \text{pmf } (?pmf\text{-fun}' xa) \text{ True}) / \text{real } (\text{card } ?\mathcal{K})$
 $=$
 $(\sum xa \in ?\mathcal{K}. (\sum xb \in (?\mathcal{K} - \{xa\}). \text{pmf } ((?pmf\text{-fun}''' xa) xb) \text{ True})) / (\text{real } (\text{card } ?\mathcal{K} - 1) * \text{real } (\text{card } ?\mathcal{K}))$
by *auto*

have $\bigwedge p \ q. (p \in ?\mathcal{K} \wedge q \in ?\mathcal{K}) \longrightarrow 2 \leq p \wedge 2 \leq q$
by *auto*

then have $\bigwedge p \ q. (p \in ?\mathcal{K} \wedge q \in ?\mathcal{K}) \longrightarrow 1 < p * q$
using *One-nat-def mem-Collect-eq one-less-mult prime-gt-1-nat*
by *(metis (mono-tags, lifting))*

then have $\bigwedge p \ q. (p \in ?\mathcal{K} \wedge q \in ?\mathcal{K}) \longrightarrow 1 < p * q \wedge \text{coprime } 1 (p * q)$
by *auto*

then have $\bigwedge p \ q. (p \in ?\mathcal{K} \wedge q \in ?\mathcal{K}) \longrightarrow 1 \in ?Z \ p \ q$
by *blast*

then have $Z\text{-not-empty}: \bigwedge p \ q. (p \in ?\mathcal{K} \wedge q \in ?\mathcal{K}) \Longrightarrow ?Z \ p \ q \neq \{\}$
by *blast*


```

have  $\mathcal{Z}$ -finite:  $\bigwedge p q. \text{finite } (?Z \ p \ q)$ 
by auto

have  $\bigwedge xa \ xb \ xc. xc \in (?Z \ xa \ xb)$ 
 $\implies \text{pmf } ((?pmf\text{-fun-4 } xa \ xb) \ xc) \ \text{True} = 1$  by auto

have  $\bigwedge xa \ xb. xa \in ?K \implies xb \in ?K - \{xa\} \implies \text{pmf } (?pmf\text{-fun-3alt } xa \ xb) \ \text{True}$ 
=
 $(\sum_{xc \in (?Z \ xa \ xb)}. \text{pmf } ((?pmf\text{-fun-4 } xa \ xb) \ xc) \ \text{True}) / \text{real } (\text{card } (?Z \ xa \ xb))$ 

using  $\mathcal{Z}$ -finite  $\mathcal{Z}$ -not-empty
by (simp add: pmf-bind-pmf-of-set)
then have  $\bigwedge xa \ xb. xa \in ?K \implies xb \in ?K - \{xa\} \implies \text{pmf } (?pmf\text{-fun-3alt } xa \ xb) \ \text{True} =$ 
 $(\sum_{xc \in (?Z \ xa \ xb)}. 1) / \text{real } (\text{card } (?Z \ xa \ xb))$ 
by auto
then have  $\bigwedge xa \ xb. xa \in ?K \implies xb \in ?K - \{xa\} \implies \text{pmf } (?pmf\text{-fun-3alt } xa \ xb) \ \text{True} =$ 
 $(\text{card } (?Z \ xa \ xb)) / (\text{card } (?Z \ xa \ xb))$ 
by auto
then have  $\bigwedge xa \ xb. xa \in ?K \implies xb \in ?K - \{xa\} \implies \text{pmf } (?pmf\text{-fun-3alt } xa \ xb) \ \text{True} = 1$ 
using  $\mathcal{Z}$ -not-empty
by auto

from ff this have
 $(\sum_{xa \in ?K}. \text{pmf } (?pmf\text{-fun}' \ xa) \ \text{True}) / \text{real } (\text{card } ?K) =$ 
 $(\sum_{xa \in ?K}. (\sum_{xb \in (?K - \{xa\})}. 1)) / (\text{real } (\text{card } ?K - 1) * \text{real } (\text{card } ?K))$ 
by auto
then have
 $(\sum_{xa \in ?K}. \text{pmf } (?pmf\text{-fun}' \ xa) \ \text{True}) / \text{real } (\text{card } ?K) =$ 
 $(\sum_{xa \in ?K}. (\text{card } ?K - 1)) / (\text{real } (\text{card } ?K - 1) * \text{real } (\text{card } ?K))$ 
by auto
then have
 $(\sum_{xa \in ?K}. \text{pmf } (?pmf\text{-fun}' \ xa) \ \text{True}) / \text{real } (\text{card } ?K) =$ 
 $((\text{card } ?K) * (\text{card } ?K - 1)) / ((\text{card } ?K - 1) * (\text{card } ?K))$ 
by auto
then have step-4:  $(\sum_{xa \in ?K}. \text{pmf } (?pmf\text{-fun}' \ xa) \ \text{True}) / \text{real } (\text{card } ?K) = 1$ 
using  $\mathcal{K}$ -not-empty  $\mathcal{K}$ -card-after-p div-self
by auto

from step-1 spmf-last
have pmf (do {
 $(n, (p, q)) \leftarrow \text{key-gen-prob } l \ u;$ 
 $(c, x) \leftarrow (\text{encrypt-prob } n \ m);$ 
 $\text{return-pmf } (\text{decrypt-alt } p \ q \ (c, x) = m)$ }) True
= pmf (do {
 $\text{let } \mathcal{K} = \{(p::\text{nat}). \text{prime } p \wedge p \bmod 4 = 3 \wedge l \leq p \wedge p \leq u\};$ 

```

```

     $p \leftarrow \text{pmf-of-set } \mathcal{K};$ 
     $q \leftarrow \text{pmf-of-set } (\mathcal{K} - \{p\});$ 
     $\text{let } \mathcal{Z} = \{(z::\text{nat}). z < p * q \wedge \text{coprime } z (p * q)\};$ 
     $r \leftarrow \text{pmf-of-set } \mathcal{Z};$ 
     $\text{return-pmf } (((p \in \mathcal{K} \wedge q \in \mathcal{K} - \{p\} \wedge r \in \mathcal{Z}) \vee (\text{prime } p \wedge \text{prime } q \wedge p \bmod 4$ 
 $= 3 \wedge q \bmod 4 = 3 \wedge r < p * q \wedge \text{coprime } r (p * q) \wedge p \neq q))$ 
 $\vee \text{decrypt-alt } p \ q \ (\text{encrypt-alt } (p * q) \ m \ r) = m))$ 
     $\text{True}$ 
    by auto

    then have  $\text{pmf } (\text{do } \{$ 
     $(n, (p, q)) \leftarrow \text{key-gen-prob } l \ u;$ 
     $(c, x) \leftarrow (\text{encrypt-prob } n \ m);$ 
     $\text{return-pmf } (\text{decrypt-alt } p \ q \ (c, x) = m)\}$   $\text{True} =$ 
     $(\sum_{xa \in ?\mathcal{K}} \text{pmf } (? \text{pmf-fun } xa) \ \text{True}) / \text{real } (\text{card } ?\mathcal{K})$ 
    using step-3 replace-pmf
    by presburger

    then show ?thesis
    using step-4
    by auto
qed

end

```